
Universidad Técnica Estatal de Quevedo

Carrera de Ingeniería de Software

GA — Práctica en Clase:

Instalación de entorno y primer script

PHP 8.2 y PERL 5.38/CGI — Procesamiento seguro de formularios web

Asignatura: Aplicaciones Web

Código / Paralelo: SOFT-R-A | 5to Nivel

Estudiante: Loor Medranda Marlon Taylor

Período: UTEQ 2026 | Corte 1

Semana: 05

Fecha: 6 de junio de 2026

Índice

1. Introducción	2
2. Instalación y verificación del entorno	2
2.1. Instalación de XAMPP 8.2.x	2
2.2. Verificación de versiones en terminal	3
3. Estructura de archivos del proyecto	4
4. Formulario HTML común — formulario.html	4
4.1. Descripción	4
4.2. Código fuente	5
4.3. Resultado en el navegador	6
5. Script PHP 8.2 — procesar.php	6
5.1. Descripción y funciones clave	6
5.2. Código fuente	6
6. Script PERL/CGI — procesar.pl	8
6.1. Descripción y funciones clave	8
6.2. Consideraciones previas en Linux/Mac	9
6.3. Código fuente	9
7. Verificación con casos de prueba	11
7.1. Caso 1 — Campos vacíos	11
7.2. Caso 2 — Email sin dominio	13
7.3. Caso 3 — Email sin arroba	14
7.4. Caso 4 — Edad negativa (-5)	16
7.5. Caso 5 — Edad demasiado alta (200)	18
7.6. Caso 6 — Edad no numérica (veinte)	20
7.7. Caso 7 — Intento XSS en el campo nombre	22
7.8. Caso 8 — Intento XSS en el campo mensaje	24
7.9. Caso 9 — Datos completamente válidos	27
7.10. Caso 10 — Nombre con caracteres especiales válidos	29
8. Tabla comparativa: PHP 8.2 vs. PERL 5.38 con CGI.pm	30
9. Reflexión final	31

1. Introducción

La presente práctica corresponde a la **Semana 05** de la asignatura *Aplicaciones Web*, encuadrada en la *Unidad II — Herramientas de Programación Web del Servidor*. El objetivo principal es introducirse en la programación del lado del servidor mediante dos tecnologías históricamente relevantes y actualmente complementarias: **PHP 8.2** y **PERL 5.38 con CGI.pm**.

“El estudiante configurará un entorno de desarrollo local (XAMPP), construirá un formulario HTML común y desarrollará dos scripts independientes — uno en PHP y otro en PERL — que procesen, validen y saneen los datos enviados por el usuario, comparando de forma directa la filosofía y los mecanismos de seguridad de ambos lenguajes.”

La actividad permite entender por qué PHP domina actualmente el 77 % del mercado de servidores con lenguaje conocido [9], mientras PERL sigue siendo la referencia en administración de sistemas y bioinformática [6]. Los tres archivos entregables son:

- `htdocs/practica5/formulario.html` — Formulario HTML común para ambos backends.
- `htdocs/practica5/procesar.php` — Script PHP con `filter_input()` y `htmlspecialchars()`
- `cgi-bin/procesar.pl` — Script PERL con `CGI::escapeHTML()` y expresiones regulares.

2. Instalación y verificación del entorno

2.1. Instalación de XAMPP 8.2.x

Se descargó XAMPP 8.2.x desde <https://www.apachefriends.org/> y se ejecutó el instalador seleccionando obligatoriamente los módulos **Apache**, **PHP** y **Perl**. Una vez completada la instalación, se inició el módulo Apache desde el panel de control [8].

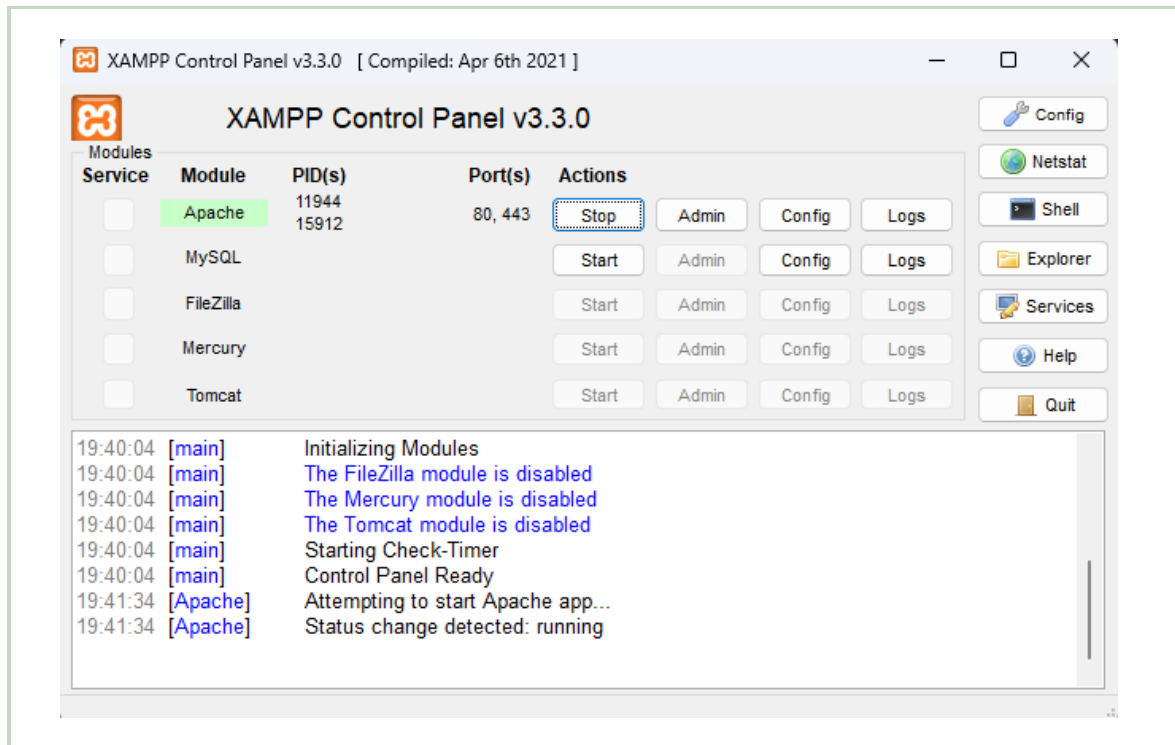


Figura 1: Panel de control de XAMPP con el módulo Apache en estado Running (fila verde), mostrando PID y puertos 80/443 activos.

2.2. Verificación de versiones en terminal

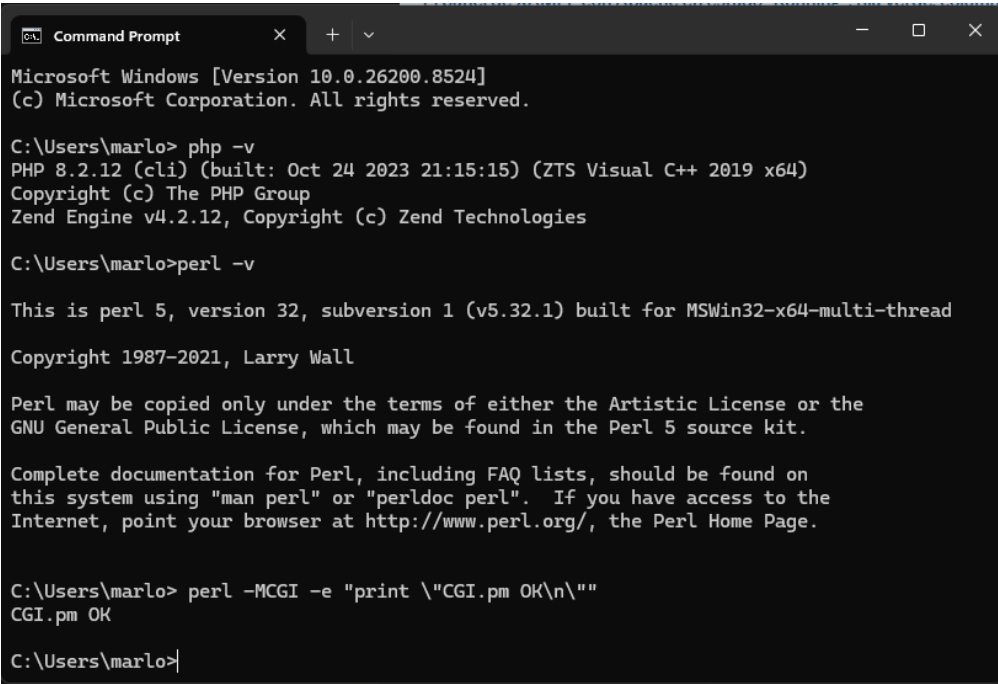
Con Apache en ejecución, se abrió una terminal y se ejecutaron los siguientes comandos de verificación [1]:

Listing 1: Comprobación de versiones instaladas

```

1 # Verificar PHP
2 php -v
3 # Resultado esperado: PHP 8.2.x (cli) (built: ...)
4
5 # Verificar Perl
6 perl -v
7 # Resultado esperado: This is perl 5, version 38 ...
8
9 # Verificar modulo CGI de Perl
10 perl -MCGI -e 'print "CGI.pm OK\n"'
11 # Resultado esperado: CGI.pm OK
12
13 # Verificar que Apache responde
14 curl http://localhost/

```



```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\marlo> php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies

C:\Users\marlo> perl -v

This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread
Copyright 1987-2021, Larry Wall

Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl".  If you have access to the
Internet, point your browser at http://www.perl.org/, the Perl Home Page.

C:\Users\marlo> perl -MCGI -e "print \"CGI.pm OK\\n\""
CGI.pm OK

C:\Users\marlo>
```

Figura 2: Terminal mostrando las salidas de `php -v`, `perl -v` y la confirmación `CGI.pm OK`.

3. Estructura de archivos del proyecto

Se creó la carpeta de trabajo `practica5/` dentro del directorio raíz de XAMPP y se distribuyeron los tres archivos según la estructura requerida:

Listing 2: Estructura de directorios del proyecto

```
1 C:\xampp\
2  htdocs\
3     practica5\
4       formulario.html    <- Formulario HTML compartido
5       procesar.php       <- Backend PHP 8.2
6   cgi-bin\
7     procesar.pl         <- Backend PERL/CGI
```

En sistemas Linux/Mac la ruta base es `/opt/lampp/` y los permisos del script PERL deben establecerse con `chmod 755 procesar.pl`.

4. Formulario HTML común — `formulario.html`

4.1. Descripción

Se construyó un único formulario HTML5 con cuatro campos (nombre, email, edad y mensaje) y dos botones de envío que utilizan el atributo `formaction` para redirigir los datos al backend correspondiente sin modificar el formulario [10].

4.2. Código fuente

Listing 3: formulario.html — Formulario compartido PHP y PERL

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Practica 5 - PHP y PERL/CGI</title>
7   <style>
8     body { font-family: Arial, sans-serif; max-width: 600px;
9           margin: 2rem auto; padding: 0 1rem; }
10    .form-group { margin-bottom: 1rem; }
11    label { display: block; font-weight: bold; margin-bottom: 4px; }
12    input, textarea { width: 100%; padding: 8px; border: 1px solid #ccc;
13                   border-radius: 4px; box-sizing: border-box; }
14    .btn-group { display: flex; gap: 12px; margin-top: 1rem; }
15    button { flex: 1; padding: 10px; border: none; border-radius: 4px;
16           cursor: pointer; font-size: 1rem; color: white; }
17    .btn-php { background: #7779b3; }
18    .btn-perl { background: #3d8b3d; }
19    button:hover { opacity: 0.88; }
20  </style>
21 </head>
22 <body>
23   <h1>Formulario de registro</h1>
24   <p>Completa todos los campos. Prueba con PHP o con PERL/CGI.</p>
25   <form id="formulario" method="POST">
26     <div class="form-group">
27       <label for="nombre">Nombre completo *</label>
28       <input type="text" id="nombre" name="nombre"
29             placeholder="Ej: Maria Garcia" maxlength="100">
30     </div>
31     <div class="form-group">
32       <label for="email">Correo electronico *</label>
33       <input type="email" id="email" name="email"
34             placeholder="usuario@dominio.com">
35     </div>
36     <div class="form-group">
37       <label for="edad">Edad (1-120) *</label>
38       <input type="number" id="edad" name="edad"
39             min="1" max="120" placeholder="Ej: 22">
40     </div>
41     <div class="form-group">
42       <label for="mensaje">Mensaje *</label>
43       <textarea id="mensaje" name="mensaje" rows="4"
44               maxlength="500" placeholder="Escribe tu mensaje aqui...">
45     </textarea>
46   </div>
47   <div class="btn-group">
48     <button type="submit" class="btn-php"
49           formaction="procesar.php">Enviar con PHP</button>
50     <button type="submit" class="btn-perl"
51           formaction="/cgi-bin/procesar.pl">Enviar con PERL/CGI</button>
52   </div>
53 </form>
54 </body>
55 </html>

```

4.3. Resultado en el navegador

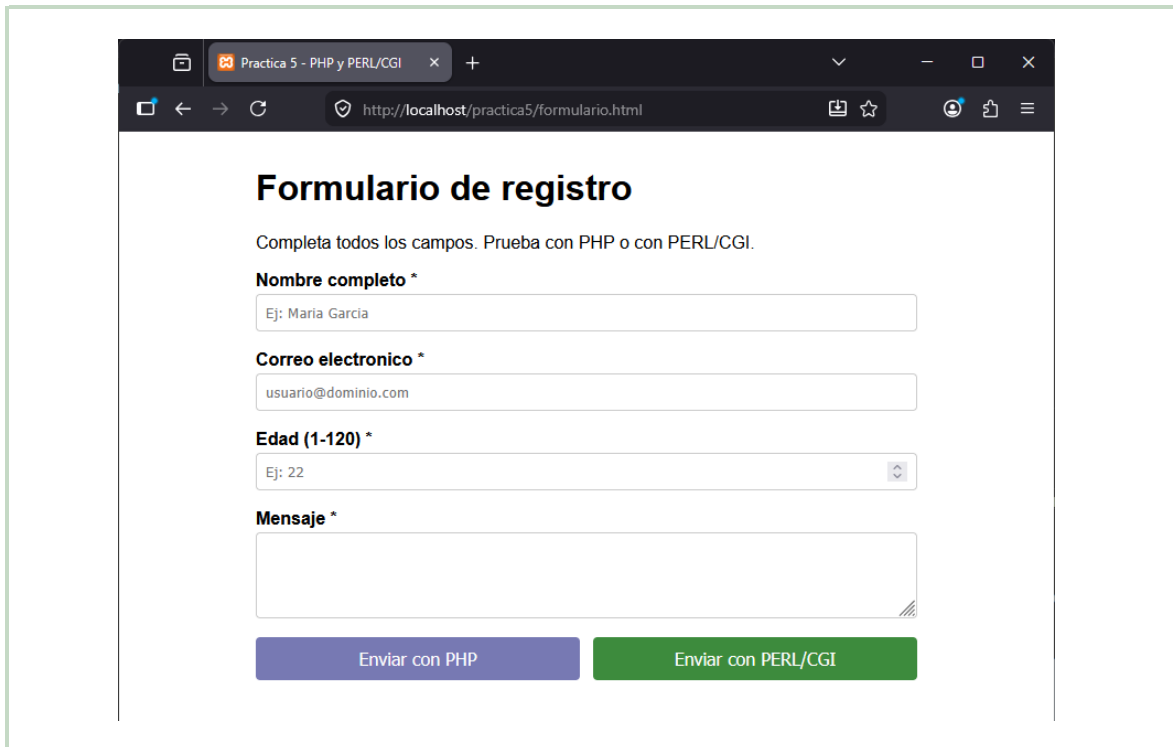


Figura 3: `formulario.html` en `http://localhost/practica5/formulario.html`, con los cuatro campos y ambos botones de envío visibles.

5. Script PHP 8.2 — `procesar.php`

5.1. Descripción y funciones clave

El script verifica que el método de envío sea `POST`, lee cada campo con `filter_input()`, aplica `htmlspecialchars()` para prevenir ataques XSS y muestra un resumen o los errores encontrados [2].

Función / Constante	Propósito
<code>filter_input()</code>	Lee una variable de entrada aplicando un filtro de validación o saneamiento. Más seguro que <code>\$_POST</code> .
<code>FILTER_VALIDATE_EMAIL</code>	Valida formato de email según RFC 5321; devuelve <code>false</code> si no es válido.
<code>FILTER_VALIDATE_INT</code>	Valida que el valor sea un entero; permite definir rango mín./máx.
<code>htmlspecialchars()</code>	Convierte <code><</code> , <code>></code> , <code>"</code> , <code>&</code> en entidades HTML, previniendo XSS.
<code>declare(strict_types=1)</code>	Activa tipos estrictos en PHP 8.

5.2. Código fuente

Listing 4: procesar.php — Validación y saneamiento seguro

```

1 <?php
2 declare(strict_types=1);
3
4 // 1. Verificar metodo HTTP
5 if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
6     header('Location: formulario.html');
7     exit;
8 }
9
10 // 2. Leer y validar cada campo con filter_input()
11 $errores = [];
12
13 // Nombre: sanear y verificar longitud
14 $nombre = filter_input(INPUT_POST, 'nombre', FILTER_DEFAULT);
15 $nombre = htmlspecialchars(trim($nombre ?? ''), ENT_QUOTES, 'UTF-8');
16 if (empty($nombre)) {
17     $errores['nombre'] = 'El nombre no puede estar vacio.';
18 } elseif (strlen($nombre) < 2 || strlen($nombre) > 100) {
19     $errores['nombre'] = 'El nombre debe tener entre 2 y 100 caracteres.';
20 }
21
22 // Email: validar formato con FILTER_VALIDATE_EMAIL
23 $email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
24 if ($email === false || $email === null) {
25     $errores['email'] = 'El correo electronico no tiene un formato valido.';
26 } else {
27     $email = htmlspecialchars($email, ENT_QUOTES, 'UTF-8');
28 }
29
30 // Edad: validar entero en rango 1-120
31 $opcionesEdad = ['options' => ['min_range' => 1, 'max_range' => 120]];
32 $edad = filter_input(INPUT_POST, 'edad', FILTER_VALIDATE_INT, $opcionesEdad);
33 if ($edad === false || $edad === null) {
34     $errores['edad'] = 'La edad debe ser un numero entero entre 1 y 120.';
35 }
36
37 // Mensaje: sanear y verificar longitud
38 $mensaje = filter_input(INPUT_POST, 'mensaje', FILTER_DEFAULT);
39 $mensaje = htmlspecialchars(trim($mensaje ?? ''), ENT_QUOTES, 'UTF-8');
40 if (empty($mensaje)) {
41     $errores['mensaje'] = 'El mensaje no puede estar vacio.';
42 } elseif (strlen($mensaje) > 500) {
43     $errores['mensaje'] = 'El mensaje no puede superar los 500 caracteres.';
44 }
45 ?>
46 <!DOCTYPE html>
47 <html lang="es">
48 <head>
49     <meta charset="UTF-8">
50     <title>Resultado PHP</title>
51     <style>
52         body { font-family: Arial, sans-serif; max-width: 600px;
53             margin: 2rem auto; padding: 0 1rem; }
54         .error { color: #c0392b; background: #fdecea; padding: 8px 12px;
55             border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
56         .exito { color: #1e6b3c; background: #eaf5ee; padding: 8px 12px;
57             border-radius: 4px; border-left: 4px solid #1e6b3c; margin: 4px 0; }

```



```

58     table { width: 100%; border-collapse: collapse; }
59     td, th { padding: 8px 12px; border: 1px solid #ddd; text-align: left; }
60     th { background: #7779b3; color: white; }
61     tr:nth-child(even) { background: #f5f5f5; }
62     .badge { display: inline-block; padding: 2px 10px; border-radius: 12px;
63             font-size: 0.8rem; background: #7779b3; color: white; }
64     </style>
65 </head>
66 <body>
67     <h1><span class="badge">PHP 8.2</span> Resultado del procesamiento</h1>
68     <?php if (!empty($errores)) : ?>
69         <h2 style="color:#c0392b;">Se encontraron errores:</h2>
70         <?php foreach ($errores as $campo => $msg) : ?>
71             <div class="error">
72                 <strong><?= htmlspecialchars(ucfirst($campo), ENT_QUOTES, 'UTF-8')
73                 ?>:</strong>
74                 <? = $msg ?>
75             </div>
76         <?php endforeach; ?>
77     <?php else : ?>
78         <div class="exito"><strong>Datos recibidos y validados
79         correctamente.</strong></div>
80         <h2>Resumen de los datos:</h2>
81         <table>
82             <tr><th>Campo</th><th>Valor recibido</th></tr>
83             <tr><td>Nombre</td><td><? = $nombre ?></td></tr>
84             <tr><td>Email</td><td><? = $email ?></td></tr>
85             <tr><td>Edad</td><td><? = $edad ?> anos</td></tr>
86             <tr><td>Mensaje</td><td><? = $mensaje ?></td></tr>
87         </table>
88     <?php endif; ?>
89     <p><a href="formulario.html">&larr; Volver al formulario</a></p>
90     <hr>
91     <p style="font-size:0.8rem;color:#888;">
92         Procesado por PHP <? = phpversion() ?> | Metodo: POST |
93         IP: <? = htmlspecialchars($_SERVER['REMOTE_ADDR'], ENT_QUOTES, 'UTF-8') ?>
94     </p>
95 </body>
</html>

```

6. Script PERL/CGI — procesar.pl

6.1. Descripción y funciones clave

El script PERL requiere que la primera línea sea el *shebang* (`#!/usr/bin/perl`) y que la cabecera HTTP se emita **antes** de cualquier salida, o Apache devolverá un error 500 [3].

Elemento	Propósito
<code>use strict;</code>	Obliga a declarar variables con <code>my</code> ; equivalente al modo estricto de JS.
<code>use warnings;</code>	Activa advertencias en tiempo de ejecución.
<code>CGI->new</code>	Encapsula la petición HTTP (equivalente a <code>\$_POST</code> en PHP).
<code>\$cgi->param()</code>	Lee el valor de un parámetro del formulario.
<code>CGI::escapeHTML()</code>	Escapa caracteres HTML; equivalente exacto de <code>htmlspecialchars()</code> en PHP.
<code>// (defined-or)</code>	Si el lado izquierdo es <code>undef</code> , usa el valor de la derecha (similar a <code>??</code> en PHP 8).
Heredoc	Bloques HTML multilínea legibles sin concatenar cadenas.

6.2. Consideraciones previas en Linux/Mac

Listing 5: Permiso de ejecución para PERL/CGI en Linux/Mac

```
1 chmod 755 /opt/lampp/cgi-bin/procesar.pl
```

En Windows con XAMPP este paso **no es necesario**.

6.3. Código fuente

Listing 6: `procesar.pl` — Validación y saneamiento en PERL/CGI

```
1 #!/usr/bin/perl
2 use strict;
3 use warnings;
4 use CGI;
5 use CGI::Carp qw(fatalsToBrowser);
6
7 # 1. Crear objeto CGI y emitir cabecera HTTP PRIMERO
8 my $cgi = CGI->new;
9 print $cgi->header(-type => 'text/html', -charset => 'UTF-8');
10
11 # 2. Leer parametros del formulario
12 my $nombre = $cgi->param('nombre') // '';
13 my $email = $cgi->param('email') // '';
14 my $edad = $cgi->param('edad') // '';
15 my $mensaje = $cgi->param('mensaje') // '';
16
17 # 3. Verificar metodo HTTP
18 my $metodo = $cgi->request_method() // '';
19 if ($metodo ne 'POST') {
20     print $cgi->redirect('formulario.html');
21     exit;
22 }
23
24 # 4. Funcion de saneamiento
25 sub sanear {
26     my ($texto) = @_;
27     $texto = CGI::escapeHTML($texto);
28     $texto =~ s/^\s+|\s+$//g;
29     return $texto;
30 }
31
```

```

32 # 5. Validaciones
33 my %errores;
34
35 $nombre = sanear($nombre);
36 if (length($nombre) < 2) {
37     $errores{nombre} = 'El nombre no puede estar vacio.';
38 } elsif (length($nombre) > 100) {
39     $errores{nombre} = 'El nombre no puede superar los 100 caracteres.';
40 }
41
42 $email = sanear($email);
43 unless ($email =~
44     /^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$/) {
45     $errores{email} = 'El correo electronico no tiene formato valido.';
46 }
47
48 $edad = sanear($edad);
49 unless ($edad =~ /\d+$/ && $edad >= 1 && $edad <= 120) {
50     $errores{edad} = 'La edad debe ser un numero entero entre 1 y 120.';
51 }
52
53 $mensaje = sanear($mensaje);
54 if (length($mensaje) == 0) {
55     $errores{mensaje} = 'El mensaje no puede estar vacio.';
56 } elsif (length($mensaje) > 500) {
57     $errores{mensaje} = 'El mensaje no puede superar los 500 caracteres.';
58 }
59
60 # 6. Generar respuesta HTML
61 my $version_perl = sprintf("%vd", $^V);
62
63 print <<HTML_INICIO;
64 <!DOCTYPE html>
65 <html lang="es">
66 <head>
67     <meta charset="UTF-8"><title>Resultado PERL/CGI</title>
68     <style>
69         body { font-family: Arial, sans-serif; max-width: 600px;
70             margin: 2rem auto; padding: 0 1rem; }
71         .error { color: #c0392b; background: #fdecea; padding: 8px 12px;
72             border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
73         .exito { color: #1e6b3c; background: #eaf5ee; padding: 8px 12px;
74             border-radius: 4px; border-left: 4px solid #1e6b3c; }
75         table { width:100%; border-collapse: collapse; }
76         td, th { padding:8px 12px; border:1px solid #ddd; }
77         th { background:#3d8b3d; color:white; }
78         tr:nth-child(even) { background:#f5f5f5; }
79         .badge { display:inline-block; padding:2px 10px; border-radius:12px;
80             font-size:0.8rem; background:#3d8b3d; color:white; }
81     </style>
82 </head>
83 <body>
84     <h1><span class="badge">PERL/CGI</span> Resultado del procesamiento</h1>
85 HTML_INICIO
86
87 if (%errores) {
88     print "<h2 style='color:#c0392b;'>Se encontraron errores:</h2>\n";
89     foreach my $campo (sort keys %errores) {

```

```

90     my $campo_display = ucfirst($campo);
91     print "<div class='error'><strong>$campo_display: </strong>"
92         . $errores{$campo} . "</div>\n";
93 }
94 } else {
95     print "<div class='exito'><strong>Datos recibidos y validados
96     correctamente.</strong></div>\n";
97     print "<h2>Resumen de los datos:</h2>\n<table>\n";
98     print "<tr><th>Campo</th><th>Valor recibido</th></tr>\n";
99     print "<tr><td>Nombre</td><td>$nombre</td></tr>\n";
100    print "<tr><td>Email</td><td>$email</td></tr>\n";
101    print "<tr><td>Edad</td><td>$edad anos</td></tr>\n";
102    print "<tr><td>Mensaje</td><td>$mensaje</td></tr>\n";
103    print "</table>\n";
104 }
105 print <<HTML_FIN;
106 <p><a href="/practica5/formulario.html">&larr; Volver</a></p>
107 <hr>
108 <p style="font-size:0.8rem;color:#888;">
109     Procesado por Perl $version_perl con CGI.pm | Metodo: POST
110 </p>
111 </body></html>
112 HTML_FIN

```

7. Verificación con casos de prueba

A continuación se documentan los diez casos de prueba ejecutados en ambos scripts, mostrando los datos de entrada, el resultado esperado y la evidencia capturada. En los casos 4, 5 y 6 se incluye una captura adicional del formulario con el dato ya ingresado (borde amarillo) para evidenciar que el valor enviado era distinto en cada prueba, dado que los tres generan el mismo mensaje de error por diseño del validador.

7.1. Caso 1 — Campos vacíos

Datos de entrada: Formulario enviado sin rellenar ningún campo.

Resultado esperado: Error en los 4 campos.

PHP 8.2

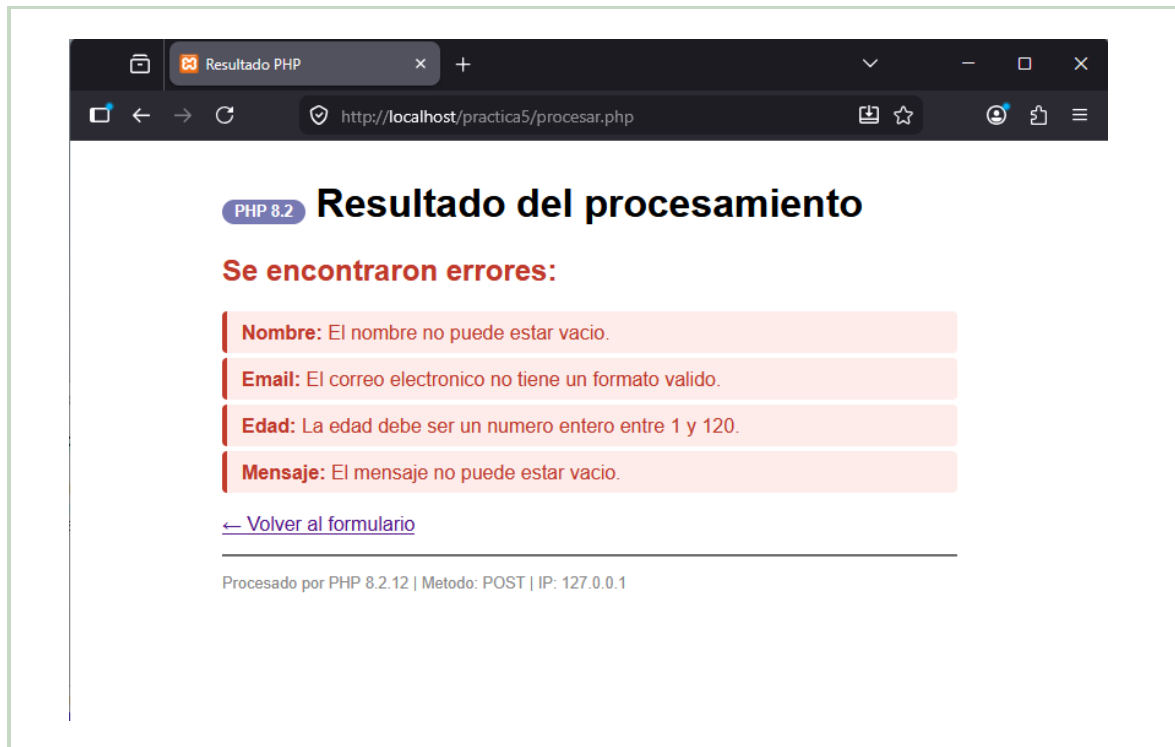


Figura 4: **PHP 8.2** — Caso 1. Los cuatro mensajes de error (nombre, email, edad y mensaje) aparecen sobre fondo rojo claro.

PERL/CGI

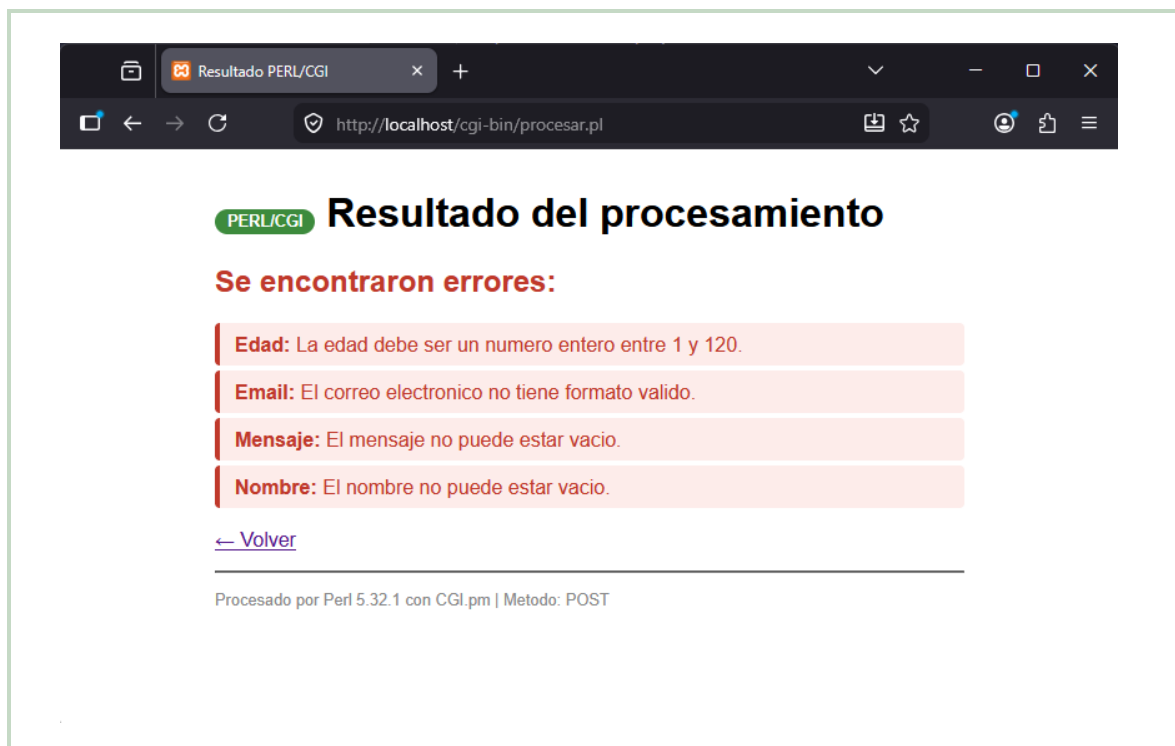


Figura 5: **PERL/CGI** — Caso 1. Idéntica respuesta con los cuatro errores, pero con el badge verde de PERL.

7.2. Caso 2 — Email sin dominio

Datos: nombre=*Ana*, email=*usuario@*, edad=*20*, mensaje=*Hola*.

Resultado esperado: Error solo en el campo email.

PHP 8.2

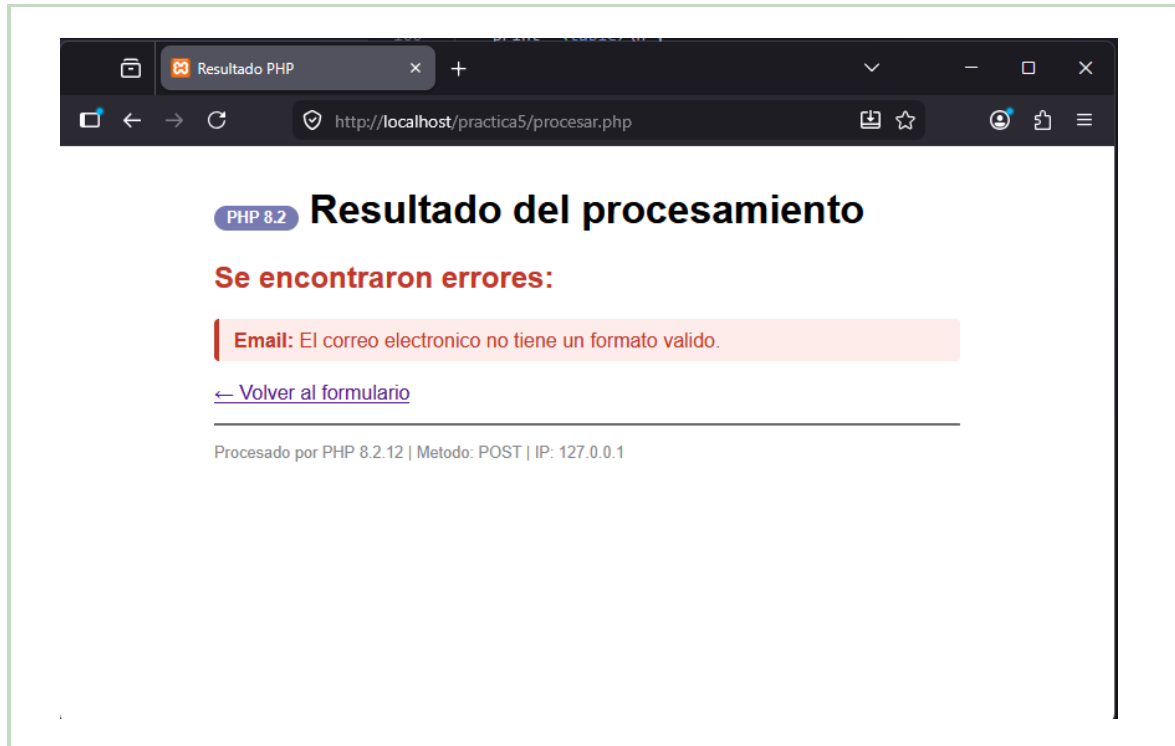


Figura 6: **PHP 8.2** — Caso 2. Un único error: “El correo electrónico no tiene un formato válido.”

PERL/CGI

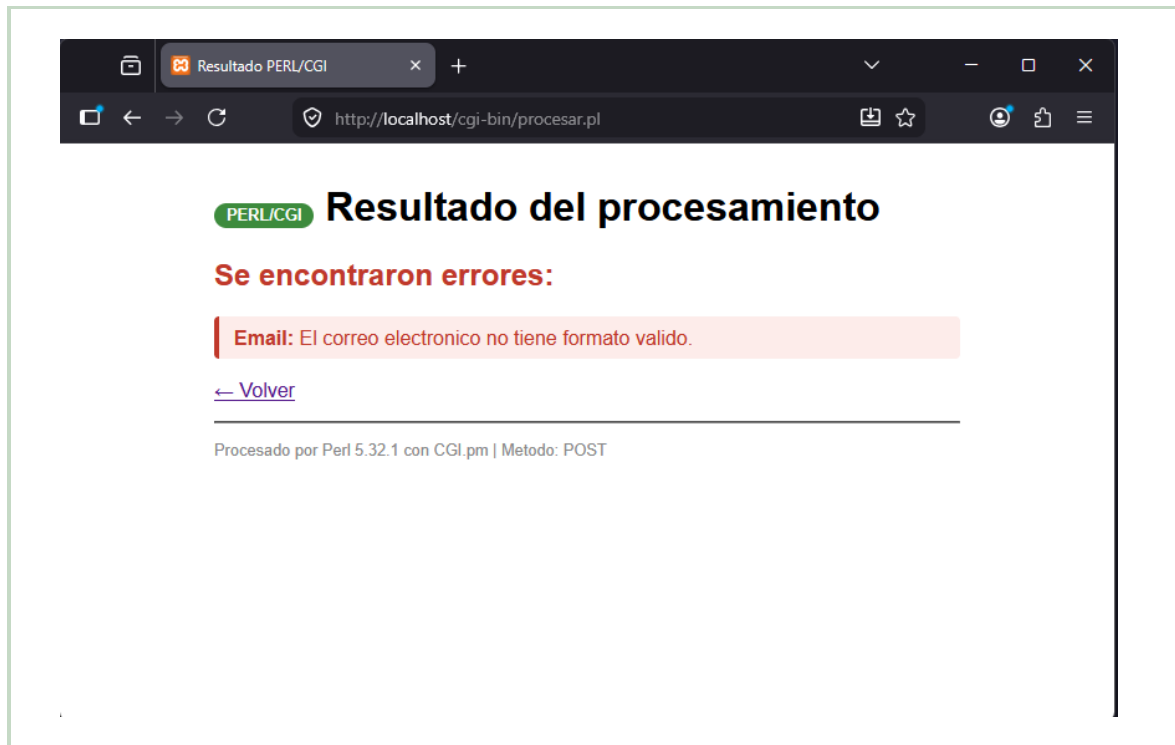


Figura 7: **PERL/CGI** — Caso 2. Mismo comportamiento validado con la expresión regular de PERL.

7.3. Caso 3 — Email sin arroba

Datos: email=usuariodominio.com (resto de campos válidos).

Resultado esperado: Error solo en email.

PHP 8.2

PERL/CGI

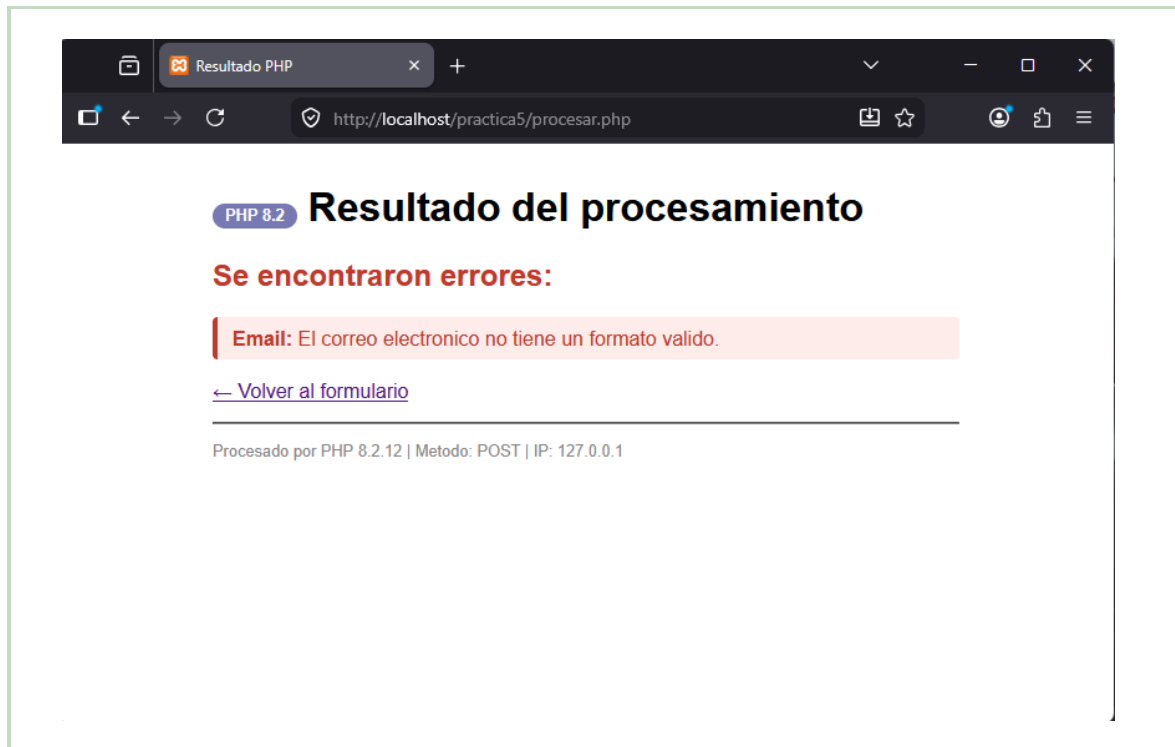


Figura 8: **PHP 8.2** — Caso 3. Error de formato de email al omitir el carácter @.

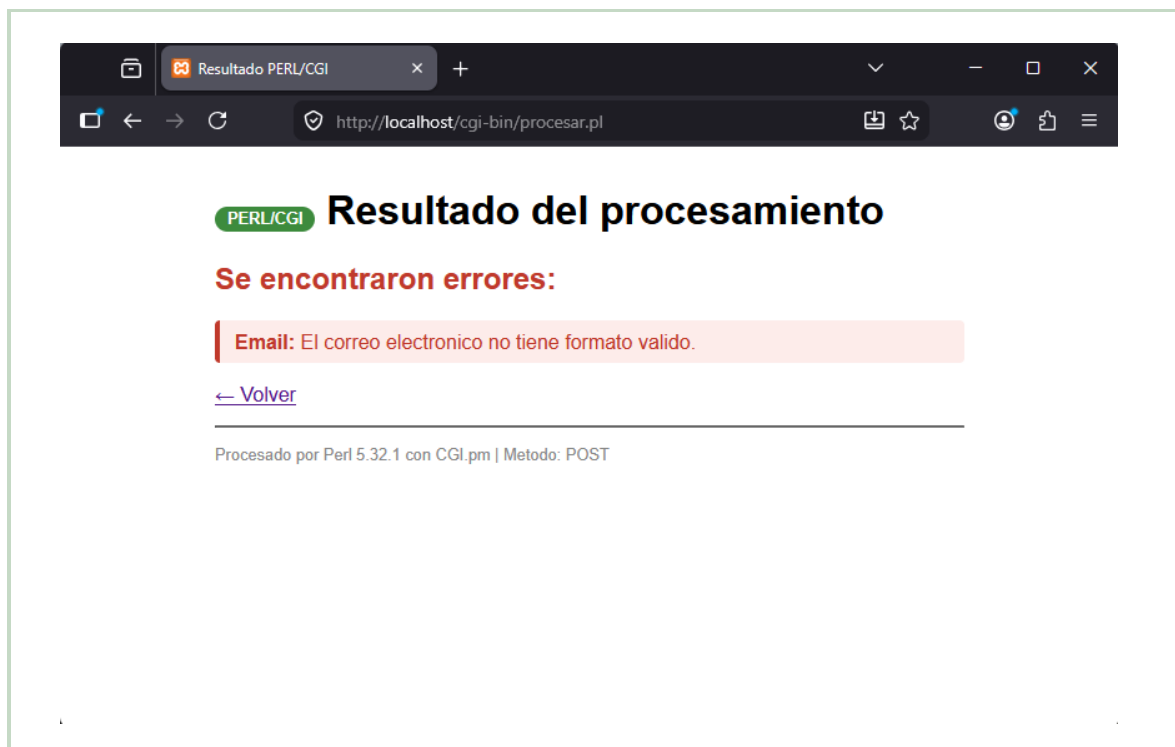
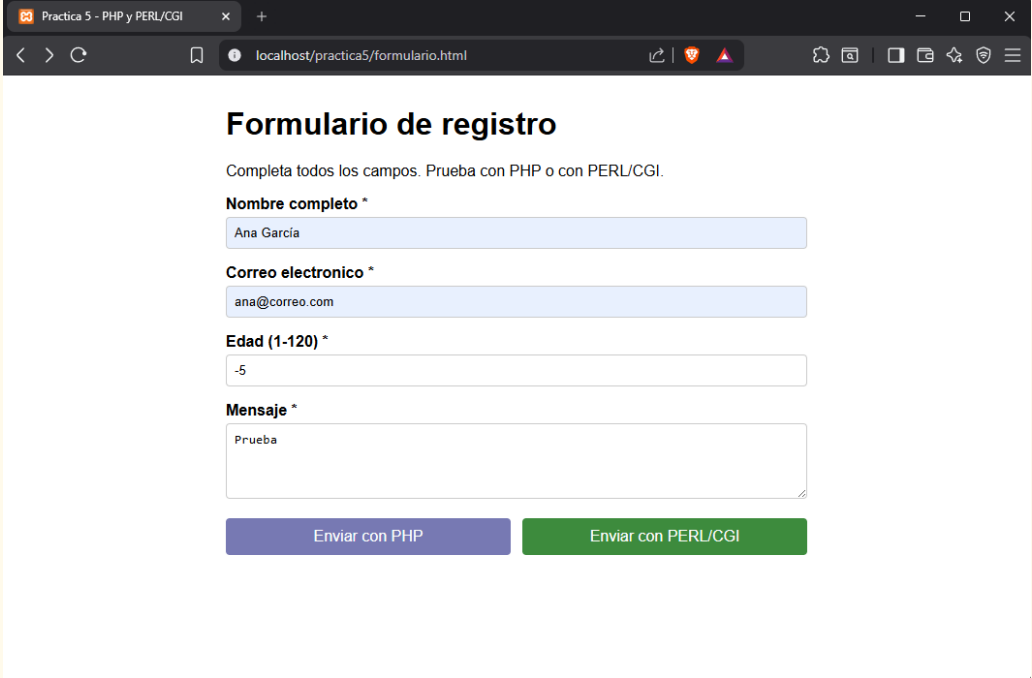


Figura 9: **PERL/CGI** — Caso 3. La expresión regular de PERL rechaza la cadena al no encontrar el símbolo @.

7.4. Caso 4 — Edad negativa (-5)

Datos: edad=-5 (resto de campos válidos).

Resultado esperado: Error solo en edad, valor inferior al mínimo permitido (1).



Practica 5 - PHP y PERL/CGI

localhost/practica5/formulario.html

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana García

Correo electronico *

ana@correo.com

Edad (1-120) *

-5

Mensaje *

Prueba

Enviar con PHP

Enviar con PERL/CGI

Dato ingresado: el campo Edad muestra -5 antes de enviar el formulario, evidenciando que el valor remitido al servidor es un entero negativo.

PHP 8.2

PERL/CGI

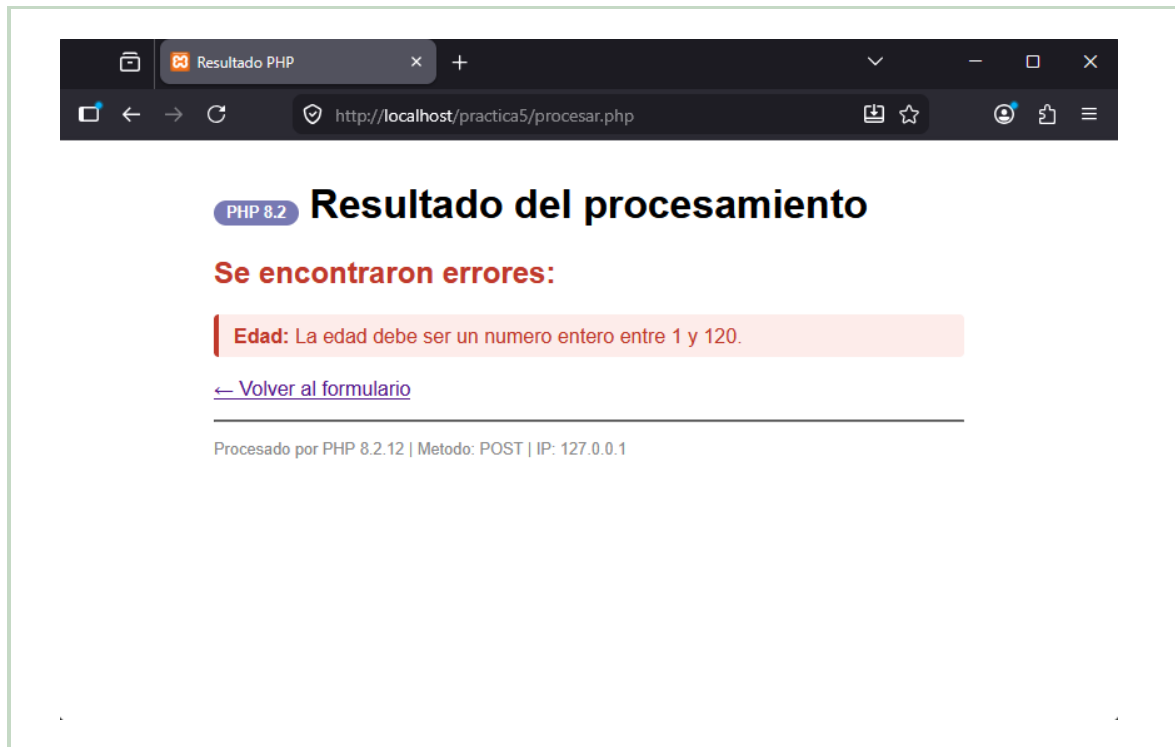


Figura 10: **PHP 8.2** — Caso 4. `FILTER_VALIDATE_INT` rechaza `-5` porque no cumple la opción `min_range = 1`.

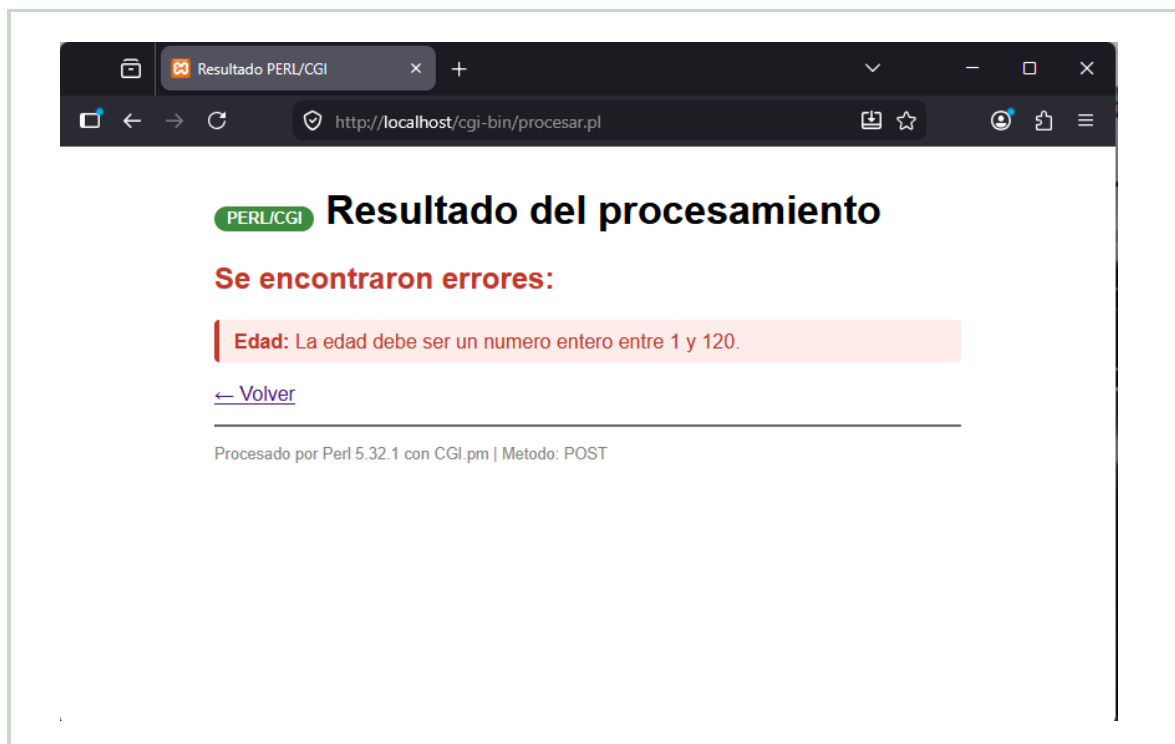
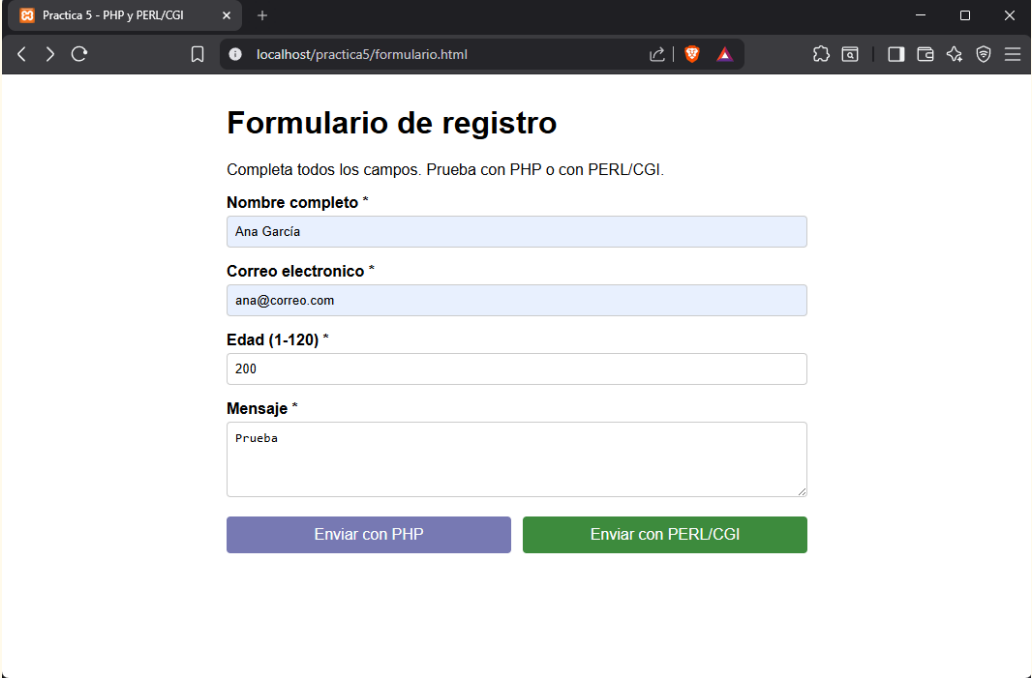


Figura 11: **PERL/CGI** — Caso 4. La condición `$edad >= 1` falla con el valor negativo.

7.5. Caso 5 — Edad demasiado alta (200)

Datos: edad=200 (resto de campos válidos).

Resultado esperado: Error solo en edad, valor superior al máximo permitido (120).



Practica 5 - PHP y PERL/CGI

localhost/practica5/formulario.html

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana García

Correo electronico *

ana@correo.com

Edad (1-120) *

200

Mensaje *

Prueba

Enviar con PHP

Enviar con PERL/CGI

Dato ingresado: el campo Edad muestra 200 antes de enviar el formulario, evidenciando que el valor remitido supera el límite máximo de 120.

PHP 8.2

PERL/CGI

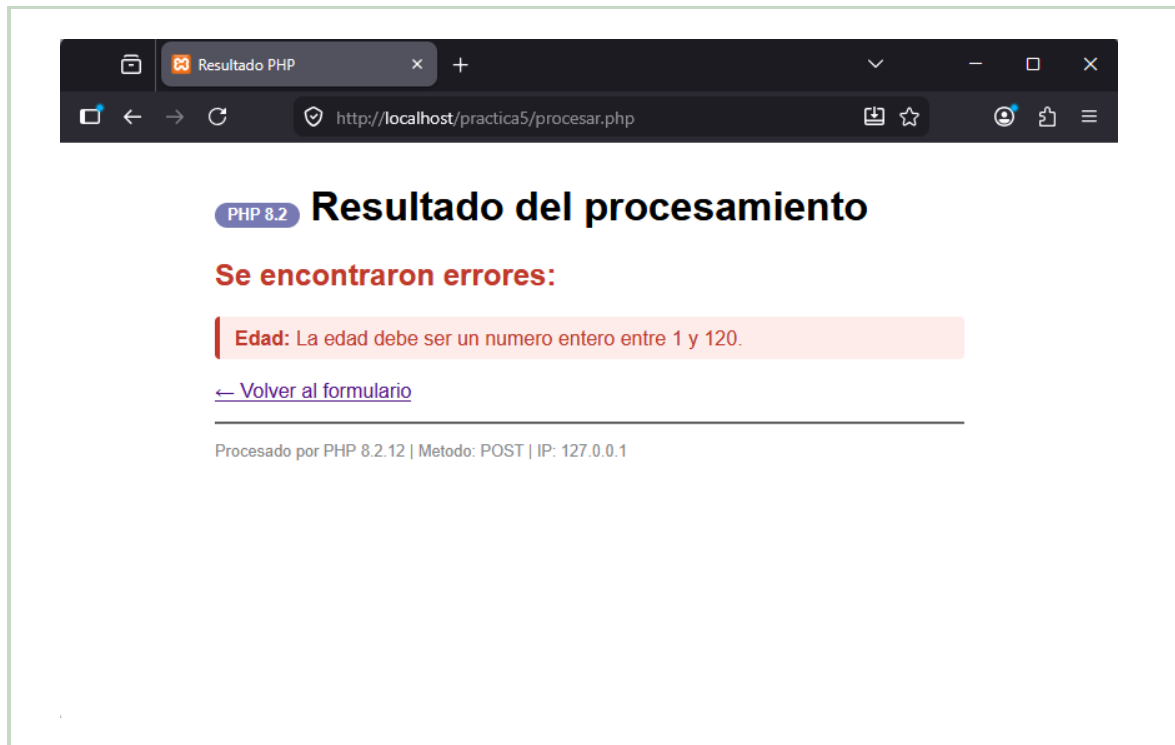


Figura 12: **PHP 8.2** — Caso 5. `FILTER_VALIDATE_INT` rechaza 200 por superar la opción `max_range = 120`.

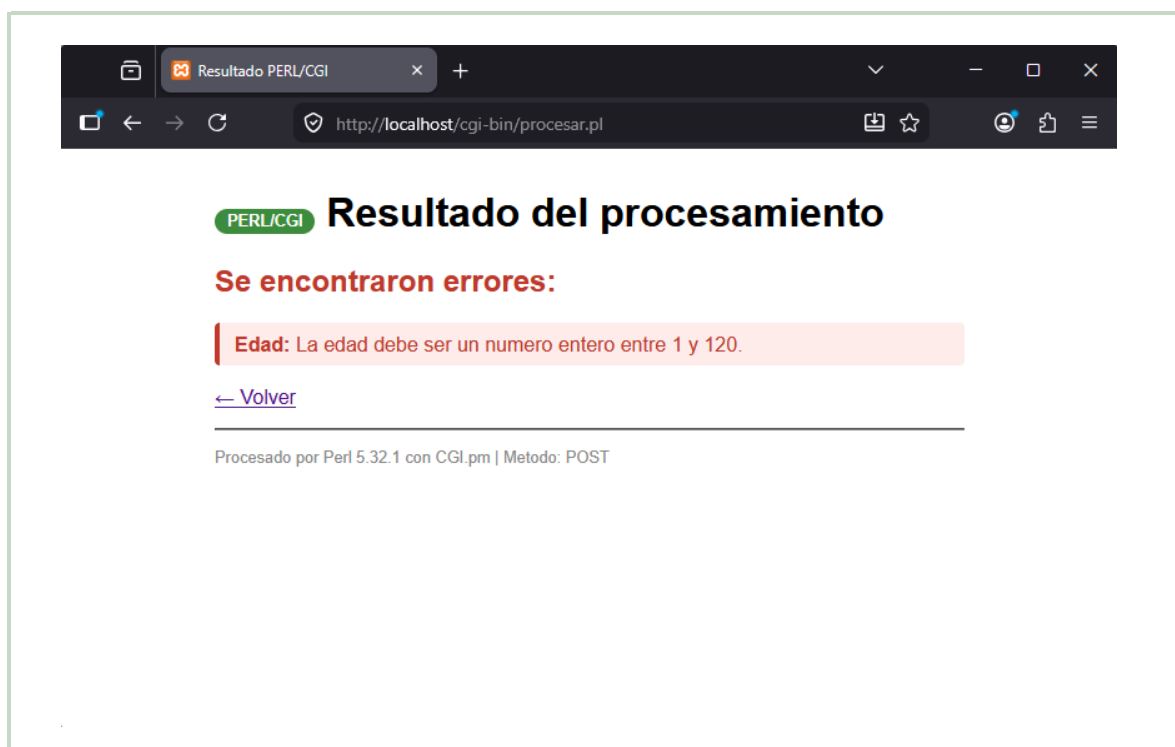
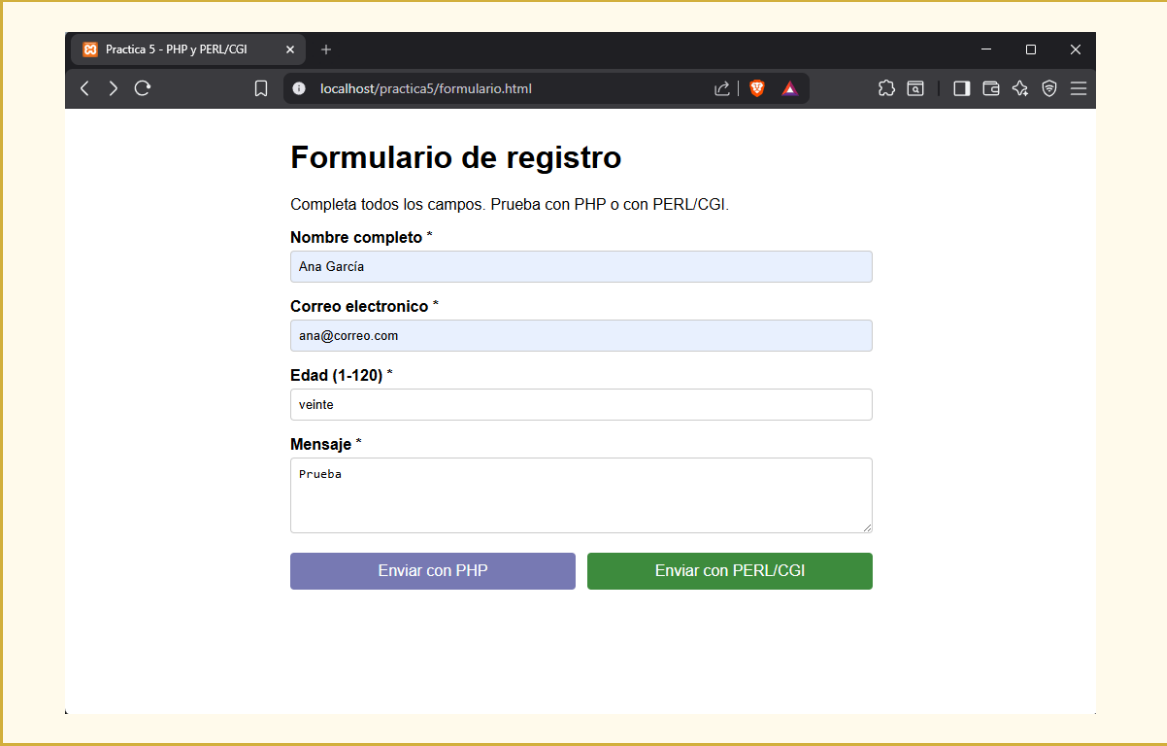


Figura 13: **PERL/CGI** — Caso 5. La condición `$edad <= 120` falla con el valor 200.

7.6. Caso 6 — Edad no numérica (veinte)

Datos: edad=veinte (resto de campos válidos).

Resultado esperado: Error solo en edad, la cadena no es un entero.



Practica 5 - PHP y PERL/CGI

localhost/practica5/formulario.html

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ana García

Correo electronico *

ana@correo.com

Edad (1-120) *

veinte

Mensaje *

Prueba

Enviar con PHP Enviar con PERL/CGI

*Dato ingresado: el campo Edad muestra la cadena **veinte** antes de enviar, evidenciando que el valor remitido es texto no numérico.*

PHP 8.2

PERL/CGI

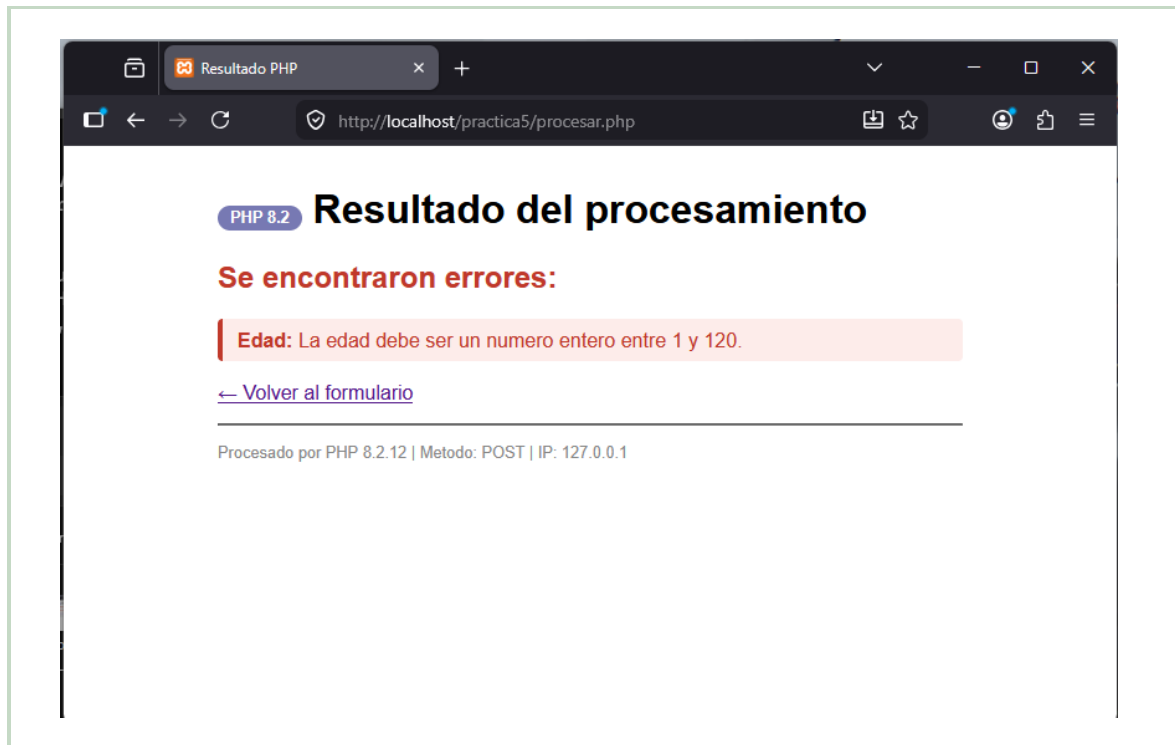


Figura 14: **PHP 8.2** — Caso 6. `FILTER_VALIDATE_INT` devuelve *false* al recibir una cadena de texto.

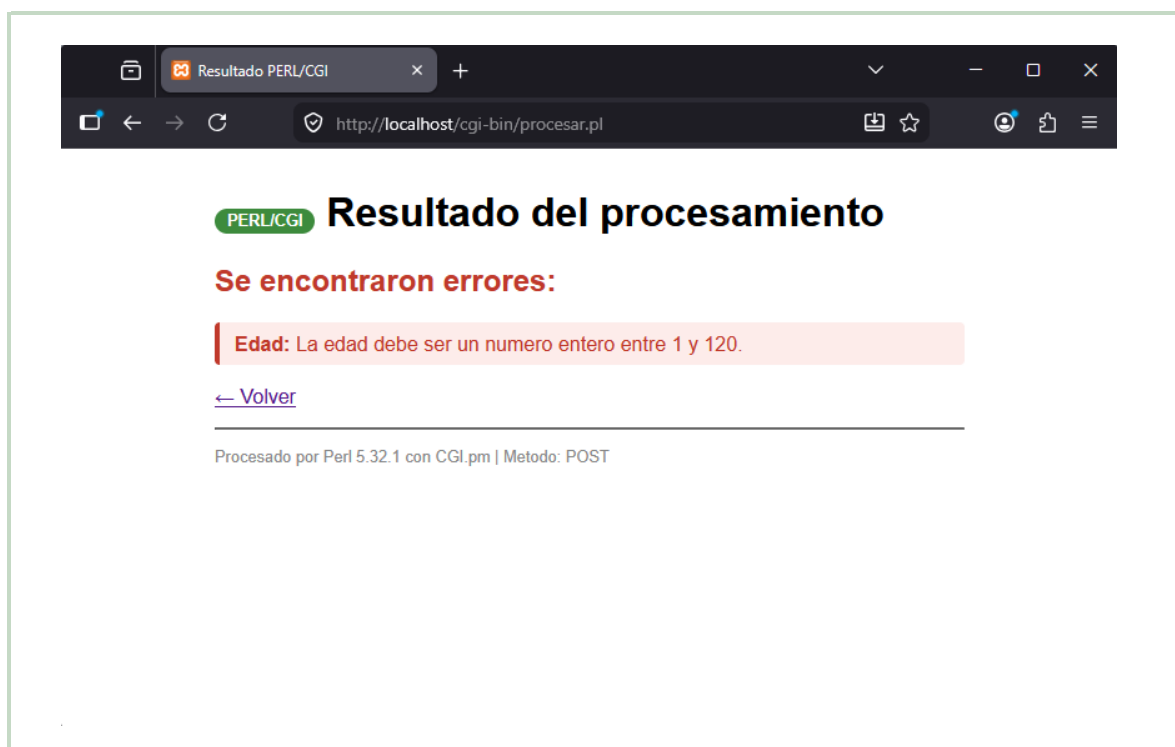


Figura 15: **PERL/CGI** — Caso 6. La expresión regular `/^\d+$/` falla porque “veinte” no contiene dígitos.

7.7. Caso 7 — Intento XSS en el campo nombre

Datos: nombre=<script>alert(1)</script> (resto válidos).

Resultado esperado: La salida muestra el texto *escapado* como <script>... sin ejecutar el código.

Importante: Se muestran dos capturas por lenguaje: el resultado en pantalla y el código fuente HTML generado (clic derecho → *Ver código fuente de la página*, o Ctrl+U), que confirma el saneamiento a nivel de entidades HTML.

PHP 8.2 — Resultado en pantalla:

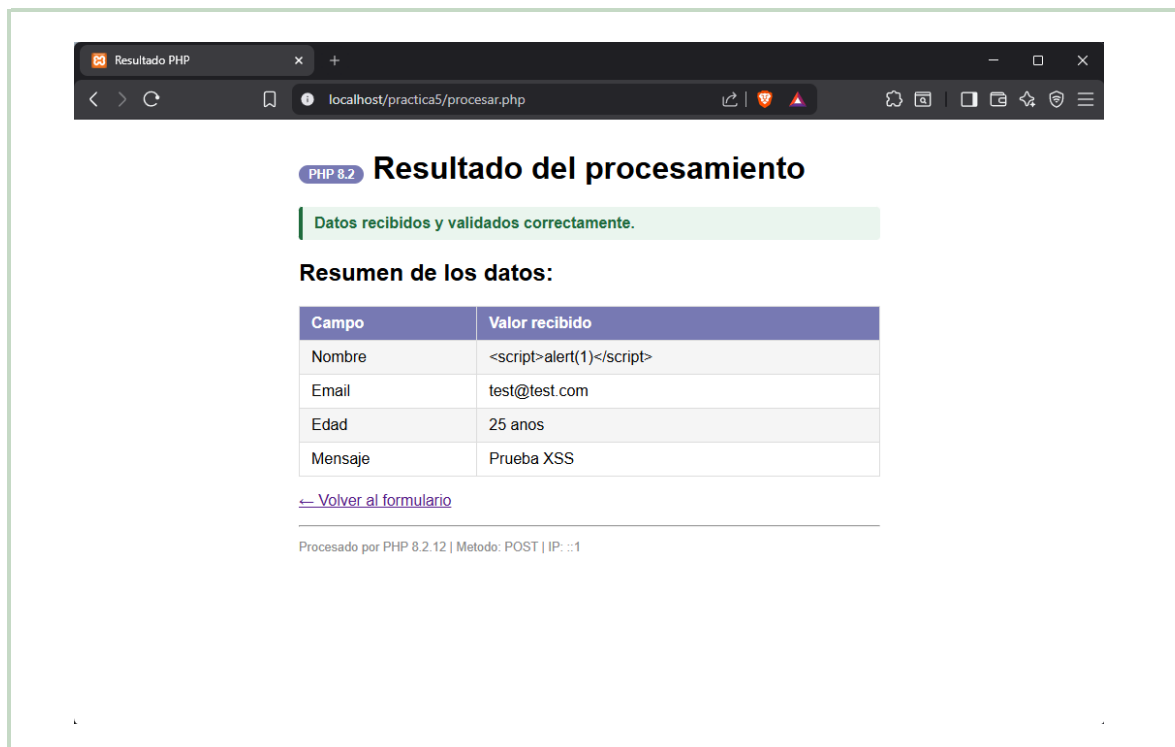


Figura 16: **PHP 8.2** — Caso 7. La etiqueta <script> se muestra como texto plano; no se ejecuta ningún alert.

PHP 8.2 — Código fuente HTML generado:

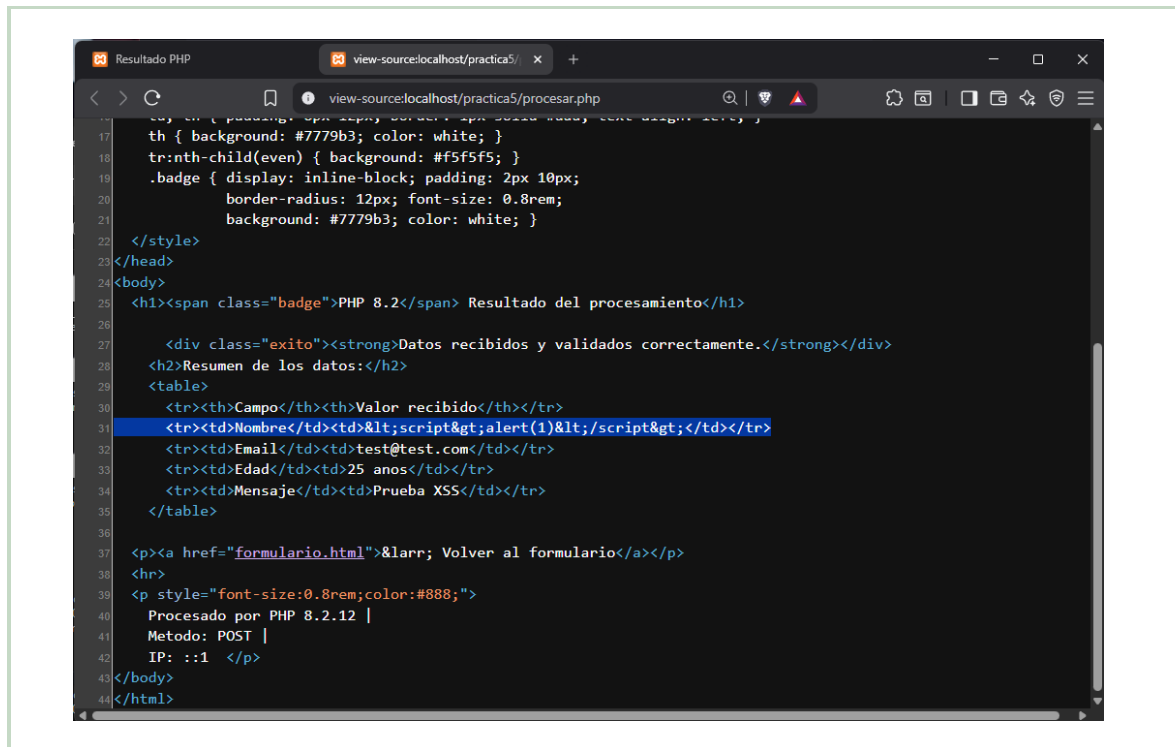


Figura 17: **PHP 8.2** — Caso 7 (código fuente). La vista Ver código fuente confirma que el input aparece como `<script>alert(1)</script>`, codificado como entidades HTML inofensivas.

PERL/CGI — Resultado en pantalla:

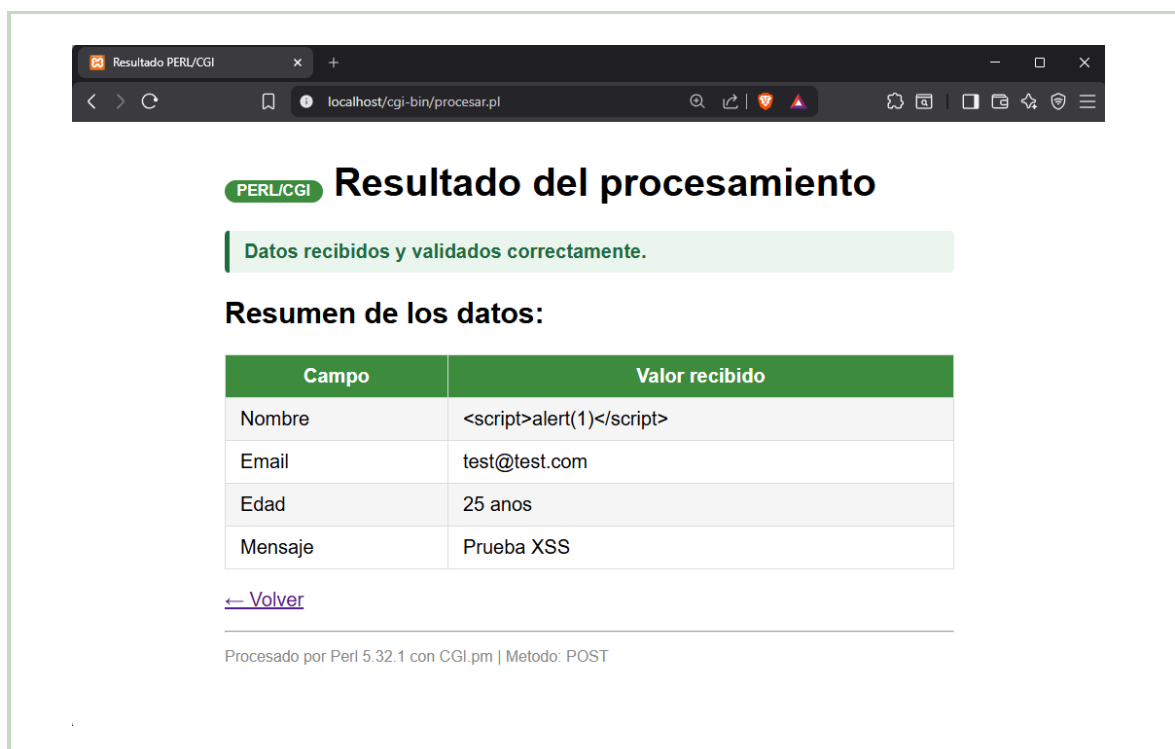


Figura 18: **PERL/CGI** — Caso 7. Mismo resultado: texto escapado visible, sin ejecución de JavaScript.

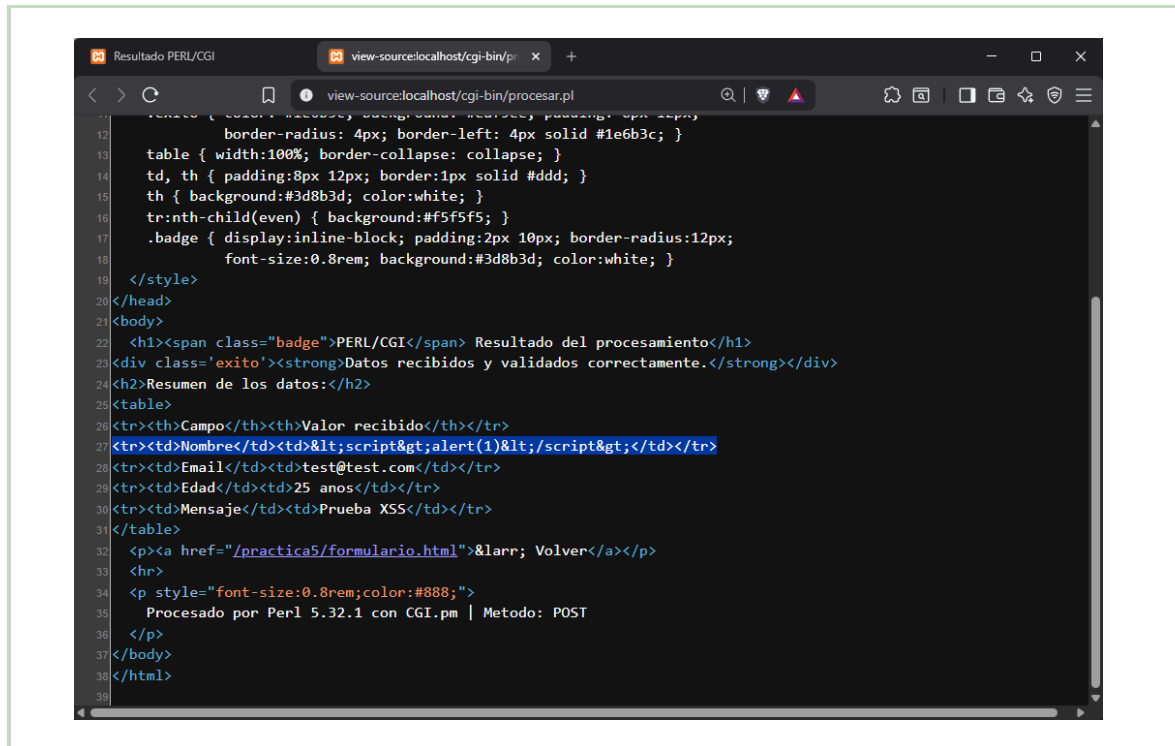
PERL/CGI — Código fuente HTML generado:

Figura 19: **PERL/CGI** — Caso 7 (código fuente). Las entidades HTML confirman el correcto saneamiento de `CGI::escapeHTML()`.

7.8. Caso 8 — Intento XSS en el campo mensaje

Datos: mensaje= (resto válidos).

Resultado esperado: La etiqueta se muestra como texto; el evento onerror no se dispara.

PHP 8.2 — Resultado en pantalla:

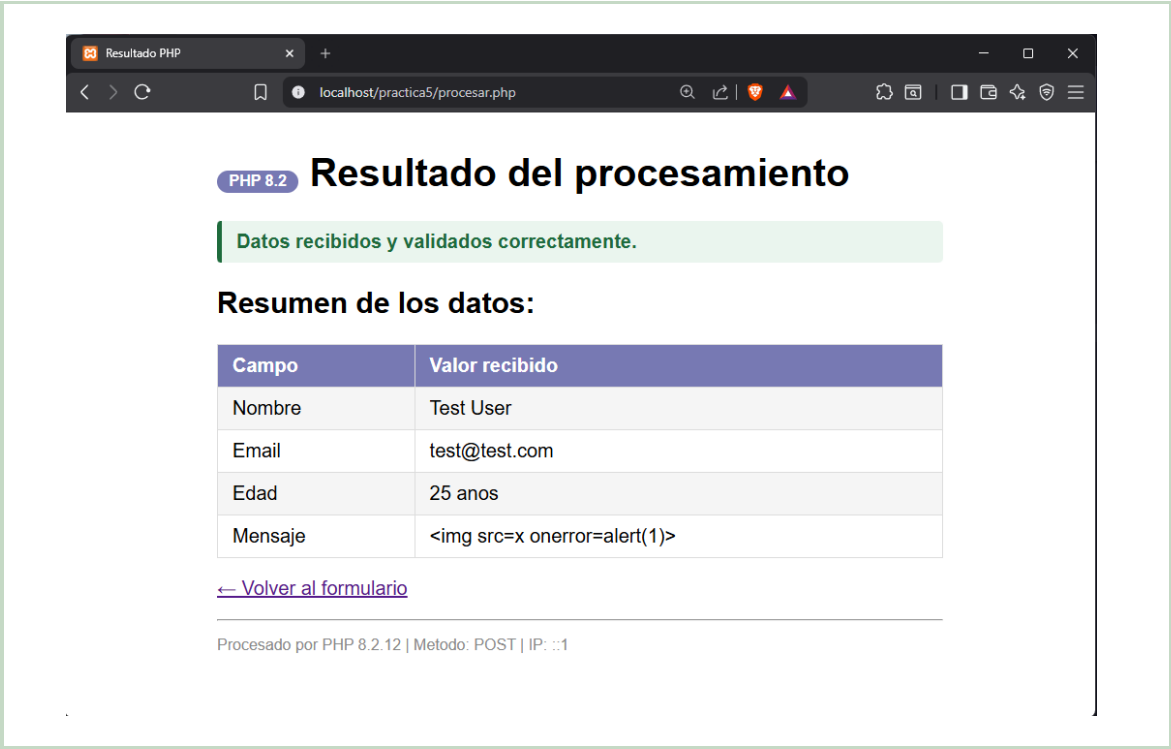


Figura 20: **PHP 8.2** — Caso 8. La etiqueta `<img>` es renderizada como texto inofensivo en la tabla de resultados.

PHP 8.2 — Código fuente HTML generado:

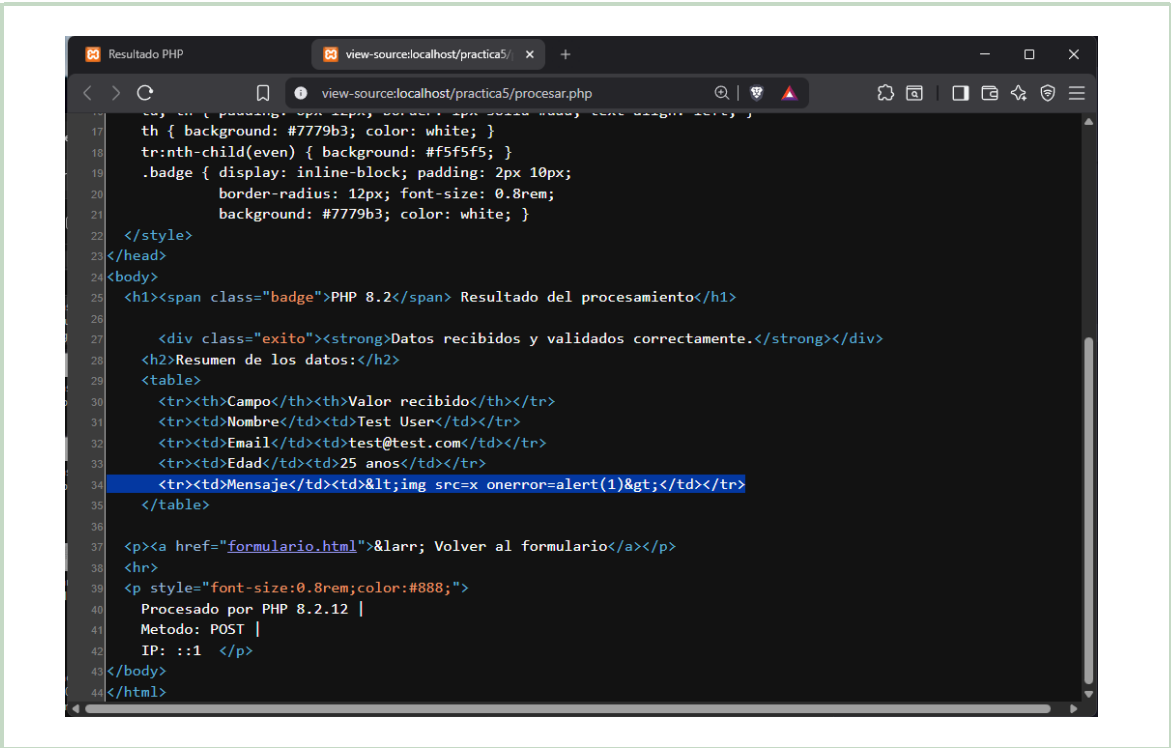


Figura 21: **PHP 8.2** — Caso 8 (código fuente). El atributo `onerror` aparece escapado como entidad, no como evento activo.

PERL/CGI — Resultado en pantalla:

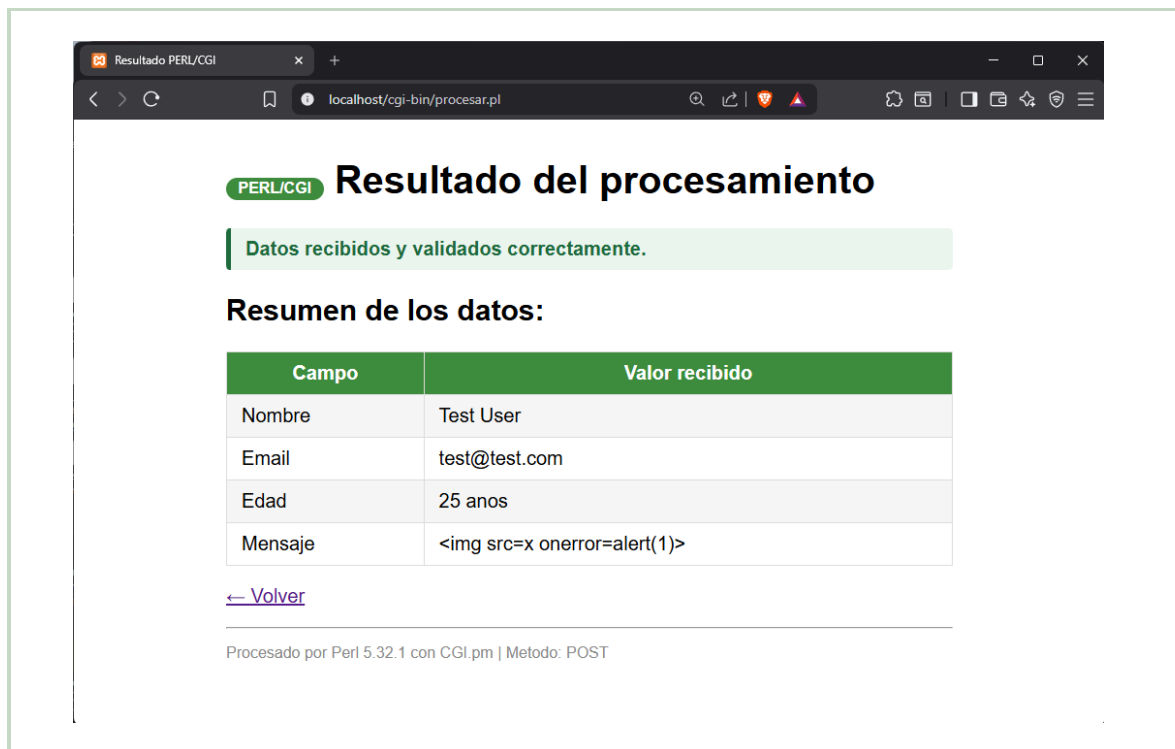


Figura 22: **PERL/CGI** — Caso 8. Resultado idéntico al de PHP: texto plano sin ejecución del evento.

PERL/CGI — Código fuente HTML generado:

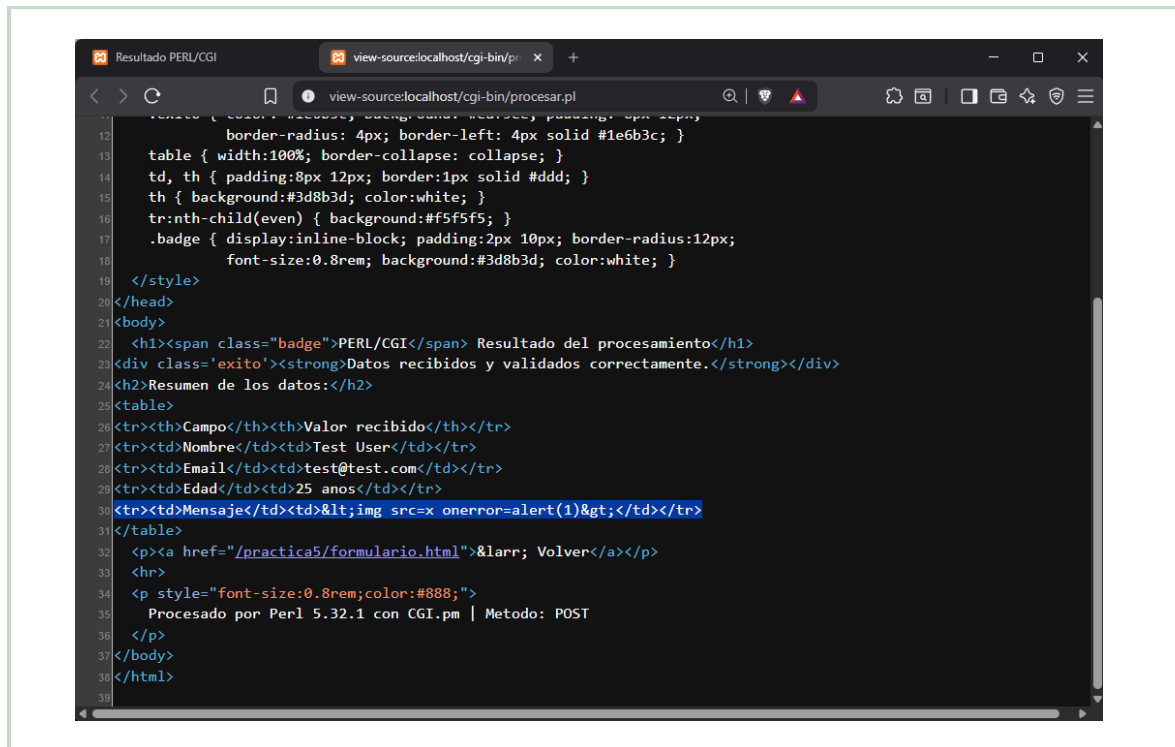


Figura 23: **PERL/CGI** — Caso 8 (código fuente). Las entidades HTML confirman el saneamiento realizado por el script Perl.

7.9. Caso 9 — Datos completamente válidos

Datos: nombre=*Juan Pérez*, email=*juan@correo.com*, edad=*25*, mensaje=*Todo funciona correctamente*.

Resultado esperado: Mensaje de éxito y tabla con los datos recibidos.

PHP 8.2

PERL/CGI

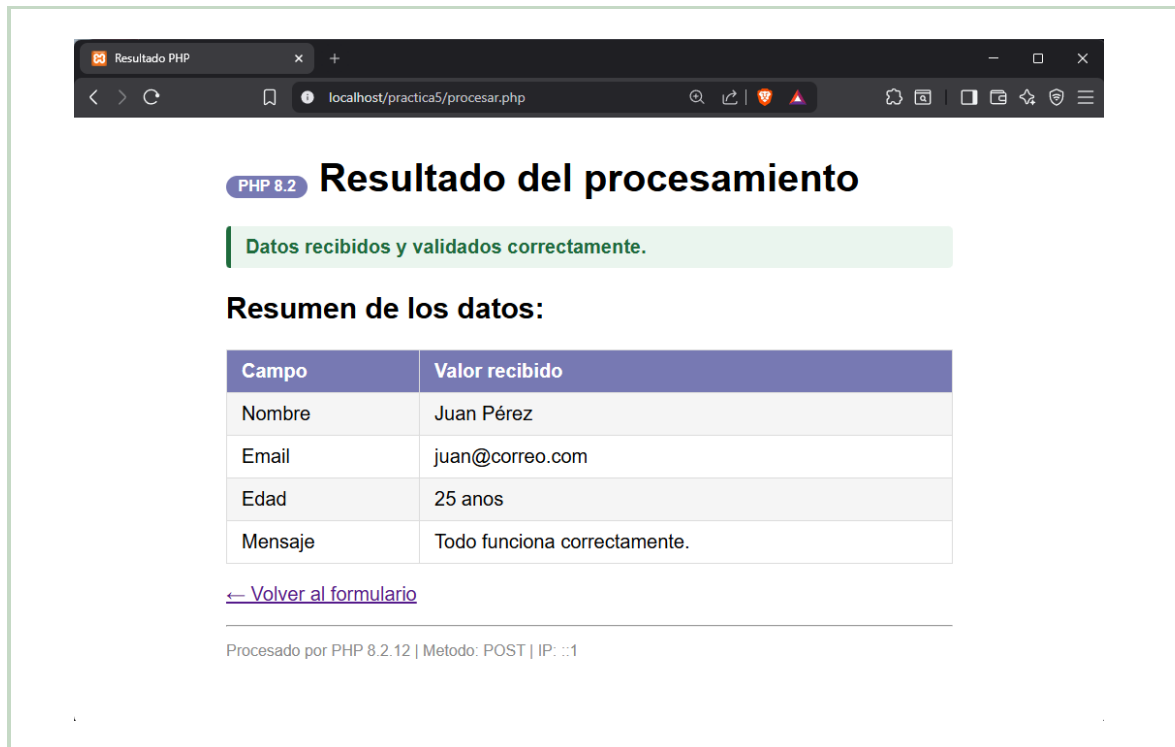


Figura 24: **PHP 8.2** — Caso 9. Banner verde de éxito y tabla con los cuatro campos recibidos y validados.

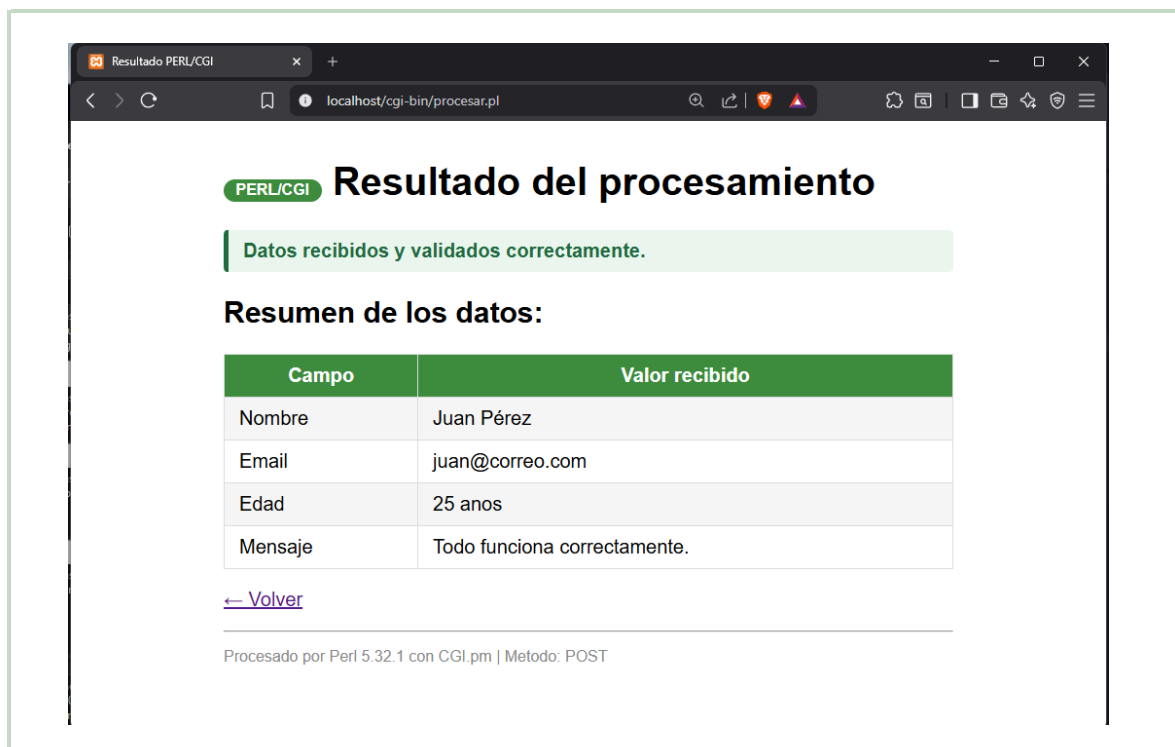


Figura 25: **PERL/CGI** — Caso 9. Mismo resultado con el badge verde; al pie se muestra la versión de Perl y CGI.pm.

7.10. Caso 10 — Nombre con caracteres especiales válidos

Datos: nombre=*María José Muñoz-López* (resto de campos válidos).

Resultado esperado: Datos aceptados y mostrados sin distorsión.

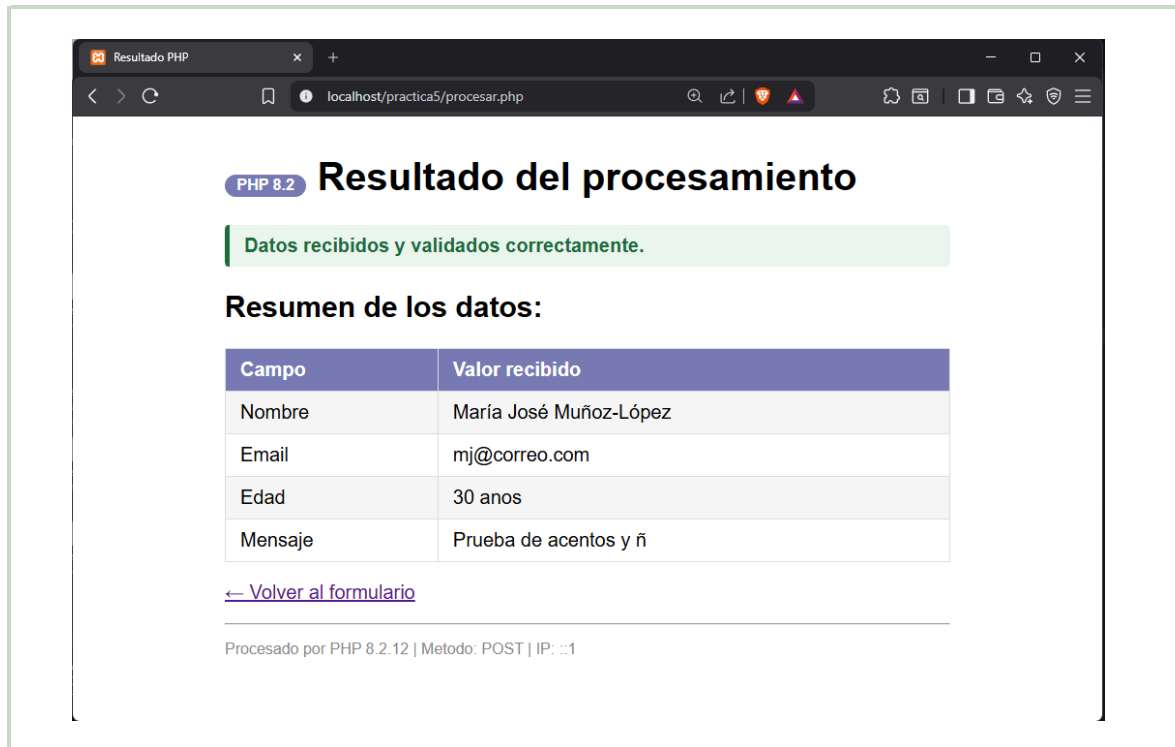
PHP 8.2**PERL/CGI**

Figura 26: **PHP 8.2** — Caso 10. El nombre con tildes, énye y guión se acepta y muestra correctamente gracias a la codificación UTF-8.

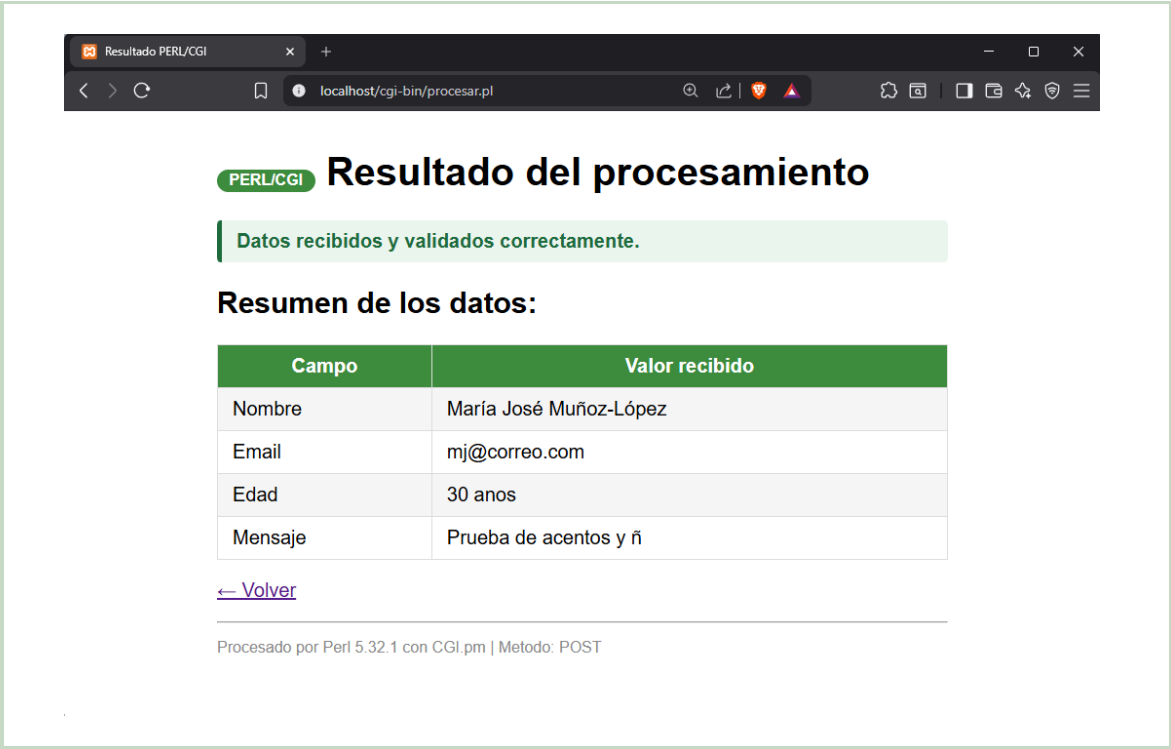


Figura 27: **PERL/CGI** — Caso 10. Mismo resultado; la cabecera `-charset =>'UTF-8'` de `CGI.pm` garantiza la correcta representación de los caracteres especiales.

8. Tabla comparativa: PHP 8.2 vs. PERL 5.38 con CGI.pm

Criterio	PHP 8.2	PERL 5.38 con CGI.pm
Integración con Apache	Módulo nativo (<code>mod_php</code>): el intérprete vive dentro del proceso de Apache; no se crea un proceso nuevo por petición.	CGI clásico: Apache lanza un proceso Perl nuevo por cada petición; mayor <i>overhead</i> bajo carga.
Leer datos del formulario	<code>filter_input(INPUT_POST, 'campo', FILTRO)</code>	<code>\$cgi->param('campo')</code> mediante objeto <code>CGI->new</code>
Saneamiento XSS	<code>htmlspecialchars(\$val, ENT_QUOTES, 'UTF-8')</code>	<code>CGI::escapeHTML(\$val)</code>
Validación de email	<code>FILTER_VALIDATE_EMAIL</code> : filtro integrado en el núcleo de PHP.	Expresión regular manual: <code>/^[...]+\@[...]+\.[a-zA-Z]{2,}\$</code>
Validación de entero	<code>FILTER_VALIDATE_INT</code> con opciones de rango.	Regex <code>/^\d+\$/</code> + comparación numérica.
Cabecera HTTP	No obligatoria manualmente; <code>header()</code> solo para redirecciones.	Obligatoria como primera salida: <code>\$cgi->header()</code> antes de cualquier <code>print</code> .
Tipos estrictos	<code>declare(strict_types=1)</code>	<code>use strict;</code> y <code>use warnings;</code>
Salida HTML	Intercalado PHP/HTML con bloques <code><?php?></code> o <code>print</code> .	<code>print</code> , <code>heredoc</code> o <code>CGI->start_html()</code> .

Criterio	PHP 8.2	PERL 5.38 con CGI.pm
Permisos de archivo	No requiere permisos especiales en Windows/XAMPP.	<code>chmod 755</code> obligatorio en Linux/Mac.
Curva de aprendizaje	Media: sintaxis intuitiva, documentación extensa en español.	Alta: sintaxis expresiva pero densa; muchas formas de hacerlo mismo.
Casos de uso actuales	Desarrollo web dinámico, CMS (WordPress, Drupal), APIs REST.	Administración de sistemas, bioinformática, automatización de texto.
Observación personal	PHP resulta más accesible para el desarrollo web gracias a sus filtros integrados y al ecosistema de <i>frameworks</i> disponibles.	PERL demuestra mayor expresividad con expresiones regulares, lo que lo hace útil en procesamiento avanzado de texto.

Cuadro 1: Comparativa directa entre PHP 8.2 y PERL 5.38/CGI.pm.

9. Reflexión final

La diferencia más significativa observada entre PHP y PERL es la **filosofía de integración con el servidor web**. PHP, ejecutándose como módulo nativo de Apache (`mod_php`), reduce el tiempo de respuesta al evitar la creación de un proceso nuevo por cada petición, lo que lo hace claramente más adecuado para aplicaciones web de tráfico moderado a alto. PERL con CGI clásico, en cambio, refleja el modelo de la web de los años 90, donde cada solicitud genera y destruye su propio proceso; esto impone un *overhead* perceptible bajo carga, aunque puede mitigarse con *FastCGI* o *Plack*.

Desde el punto de vista de la seguridad, ambos lenguajes ofrecen mecanismos equivalentes para prevenir XSS: `htmlspecialchars()` en PHP y `CGI::escapeHTML()` en PERL. La diferencia radica en que PHP incluye filtros de validación integrados en el núcleo del lenguaje (`filter_input()`), lo que simplifica el código y reduce el margen de error humano al no tener que escribir expresiones regulares manualmente.

En un proyecto real, se elegiría PHP 8.2 para el desarrollo de una API REST o un CMS, donde la velocidad, el ecosistema de *frameworks* (Laravel, Symfony) y la documentación abundante son prioritarios. Se optaría por PERL en scripts de administración de sistemas, procesamiento masivo de ficheros de texto o tareas de bioinformática, donde su potencia con expresiones regulares y su presencia histórica en entornos Unix son ventajas concretas.

Esta práctica confirma además que los principios de seguridad — verificar el método HTTP, sanear toda entrada antes de mostrarla y validar rangos numéricos — son universales e independientes del lenguaje de backend utilizado.

Referencias

-
- [1] The PHP Group, “PHP Manual,” 2024. [En línea]. Disponible en: <https://www.php.net/manual/es/>. [Accedido: jun. 2026].
 - [2] The PHP Group, “`filter_input` — Manual de PHP,” 2024. [En línea]. Disponible en: <https://www.php.net/manual/es/function.filter-input.php>. [Accedido: jun. 2026].
 - [3] The Perl Foundation, “CGI — Simple Common Gateway Interface Class,” *MetaCPAN*, 2024. [En línea]. Disponible en: <https://metacpan.org/pod/CGI>. [Accedido: jun. 2026].
 - [4] OWASP Foundation, “OWASP Top 10: 2021,” 2021. [En línea]. Disponible en: <https://owasp.org/Top10/>. [Accedido: jun. 2026].
 - [5] R. Nixon, *Learning PHP, MySQL & JavaScript*, 6.a ed. Sebastopol, CA: O’Reilly Media, 2021, ISBN: 978-1-492-06229-9.
 - [6] R. L. Schwartz, b. d. foy y T. Phoenix, *Learning Perl: Making Easy Things Easy and Hard Things Possible*, 8.a ed. Sebastopol, CA: O’Reilly Media, 2021, ISBN: 978-1-492-09495-1.
 - [7] L. Welling y L. Thomson, *PHP and MySQL Web Development*, 5.a ed. Upper Saddle River, NJ: Addison-Wesley, 2016, ISBN: 978-0-321-83389-1.
 - [8] Apache Friends, “XAMPP — Apache + MariaDB + PHP + Perl,” 2024. [En línea]. Disponible en: <https://www.apachefriends.org/>. [Accedido: jun. 2026].
 - [9] W3Techs, “Usage Statistics of Server-Side Programming Languages for Websites,” 2024. [En línea]. Disponible en: https://w3techs.com/technologies/overview/programming_language. [Accedido: jun. 2026].
 - [10] MDN Web Docs, “HTML: HyperText Markup Language,” Mozilla Foundation, 2024. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/HTML>. [Accedido: jun. 2026].